

Contents

Módulo 4 - Persistencia de datos con JPA / Hibernate	4
Punto 4.1 - Introducción a JPA e Hibernate	4
Objetivos de aprendizaje	4
Bloque 1 - El problema de la persistencia	4
T1.1 - Qué significa persistir datos	4
T1.2 - Qué es JPA	4
T1.3 - Qué es Hibernate y qué es Spring Data JPA	5
Bloque 2 - Configuración de Spring Boot para JPA	5
T2.1 - La dependencia spring-boot-starter-data-jpa	5
T2.2 - Configuración en application.properties	6
T2.3 - Perfiles para desarrollo y producción	6
Bloque 3 - Entidades JPA	7
T3.1 - Qué es una entidad	7
T3.2 - Anotaciones básicas	7
T3.3 - Ejemplo completo de entidad	8
Bloque 4 - Repositorios Spring Data JPA	9
T4.1 - La interfaz JpaRepository	9
T4.2 - Consultas personalizadas con @Query	9
T4.3 - Paginación y ordenación	10
Bloque 5 - Transacciones y buenas prácticas	10
T5.1 - Qué es una transacción	10
T5.2 - Buenas prácticas con JPA	11
T5.3 - El problema N+1	11
Resumen de la teoría	11
Criterio profesional añadido	11
Punto 4.2 - Definición de entidades	12
Objetivos de aprendizaje	12
Bloque 1 - Qué es una entidad y su ciclo de vida	12
T1.1 - El ciclo de vida de una entidad	12
T1.2 - Requisitos de una clase entidad	12
T1.3 - Entidad vs DTO: la separación definitiva	13
Bloque 2 - Generación de identificadores	13
T2.1 - La anotación @Id	13
T2.2 - Estrategias de generación	13
T2.3 - IDs en la API	14
Bloque 3 - Mapeo de columnas	14
T3.1 - La anotación @Column	14
T3.2 - Tipos de datos Java a SQL	15
T3.3 - Tipos especiales	15
Bloque 4 - Anotaciones avanzadas	16
T4.1 - @Transient	16
T4.2 - @Embedded y @Embeddable	16
T4.3 - @Version y bloqueo optimista	17
Bloque 5 - Buenas prácticas y errores comunes	18
T5.1 - Buenas prácticas en el diseño de entidades	18
T5.2 - equals y hashCode en entidades JPA	18
T5.3 - Errores comunes al definir entidades	19
Resumen de la teoría	20
Criterio profesional añadido	20
Punto 4.3 - Repositorios JPA	20
Objetivos de aprendizaje	20
Bloque 1 - La jerarquía de repositorios	21

T1.1 - Por qué hay varias interfaces	21
T1.2 - Repository y CrudRepository	21
T1.3 - PagingAndSortingRepository y JpaRepository	21
Bloque 2 - Métodos derivados	22
T2.1 - Qué son y cómo se generan	22
T2.2 - Operadores y combinaciones	22
T2.3 - Limitaciones de los métodos derivados	23
Bloque 3 - Consultas con @Query	24
T3.1 - JPQL con @Query	24
T3.2 - Consultas de actualización y borrado	24
T3.3 - SQL nativo con @Query	25
Bloque 4 - Paginación y ordenación	25
T4.1 - Pageable, Page y Sort	25
T4.2 - Cómo se usa en el repositorio	26
T4.3 - Buenas prácticas con paginación	26
Bloque 5 - Consultas dinámicas con Specifications	27
T5.1 - El problema de las consultas dinámicas	27
T5.2 - Cómo se construyen	27
T5.3 - Cuándo usar cada enfoque	27
Resumen de la teoría	28
Criterio profesional añadido	28
Punto 4.4 - Relaciones entre entidades: OneToMany, ManyToOne	28
Objetivos de aprendizaje	28
Bloque 1 - Los cuatro tipos de relaciones	28
T1.1 - Cardinalidad de las relaciones	28
T1.2 - Cómo se mapean a la base de datos	29
T1.3 - Lado propietario y lado inverso	29
Bloque 2 - @ManyToOne	30
T2.1 - Estructura de @ManyToOne	30
T2.2 - FetchType: EAGER vs LAZY	30
T2.3 - @JoinColumn en detalle	31
Bloque 3 - @OneToMany	31
T3.1 - Estructura de @OneToMany	31
T3.2 - mappedBy y el lado inverso	31
T3.3 - Métodos helper para mantener la relación	32
Bloque 4 - Cascade y orphanRemoval	33
T4.1 - CascadeType	33
T4.2 - orphanRemoval	33
T4.3 - Cuándo usar cascade y orphanRemoval	33
Bloque 5 - Rendimiento y problemas comunes	34
T5.1 - El problema N+1	34
T5.2 - LazyInitializationException	34
T5.3 - Buenas prácticas con relaciones	35
Resumen de la teoría	35
Criterio profesional añadido	35
Punto 4.5 - Consultas básicas	35
Objetivos de aprendizaje	35
Bloque 1 - JPQL en profundidad	36
T1.1 - Qué es JPQL y su relación con SQL	36
T1.2 - Estructura de una consulta JPQL	36
T1.3 - Parámetros posicionales y nombrados	37
Bloque 2 - Joins en JPQL	37
T2.1 - Tipos de joins	37
T2.2 - JOIN vs JOIN FETCH	38
T2.3 - Cuándo usar cada tipo de join	38

Bloque 3 - Funciones y expresiones	39
T3.1 - Funciones de string	39
T3.2 - Funciones numéricas y de fecha	39
T3.3 - Expresiones condicionales	40
Bloque 4 - Agregaciones y proyecciones	40
T4.1 - Funciones de agregación	40
T4.2 - GROUP BY y HAVING	41
T4.3 - Proyecciones: devolver DTOs desde consultas	41
Bloque 5 - Buenas prácticas y errores	42
T5.1 - Buenas prácticas con consultas JPQL	42
T5.2 - Errores comunes	42
T5.3 - Rendimiento y optimización	43
Resumen de la teoría	43
Criterio profesional añadido	44
Punto 4.6 - Gestión de transacciones	44
Objetivos de aprendizaje	44
Bloque 1 - Qué es una transacción	44
T1.1 - Definición y propiedades ACID	44
T1.2 - Transacciones en Spring	44
T1.3 - @Transactional en detalle	45
Bloque 2 - Propagación	45
T2.1 - Qué es la propagación	45
T2.2 - Tipos de propagación	46
T2.3 - Cuándo usar cada propagación	46
Bloque 3 - Aislamiento y concurrencia	47
T3.1 - Niveles de aislamiento	47
T3.2 - Problemas de concurrencia	47
T3.3 - Bloqueo optimista vs pesimista	47
Bloque 4 - Rollback y excepciones	48
T4.1 - Rollback por defecto	48
T4.2 - rollbackFor y noRollbackFor	48
T4.3 - Excepciones checked vs unchecked	48
Bloque 5 - Buenas prácticas y errores	49
T5.1 - Dónde poner @Transactional	49
T5.2 - Errores comunes	49
T5.3 - Rendimiento	50
Resumen de la teoría	50
Criterio profesional añadido	51
Punto 4.7 - Testing de JPA con @DataJpaTest	51
Objetivos de aprendizaje	51
Bloque 1 - Por qué testear JPA es distinto	51
T1.1 - El problema de testear la persistencia	51
T1.2 - La pirámide de tests aplicada a JPA	51
T1.3 - Qué se prueba en un test de JPA	52
Bloque 2 - @DataJpaTest en profundidad	52
T2.1 - Qué es @DataJpaTest	52
T2.2 - La transacción por test	52
T2.3 - TestEntityManager	53
Bloque 3 - Probar consultas	53
T3.1 - Probar métodos derivados	53
T3.2 - Probar consultas con @Query	54
T3.3 - Probar Specifications	55
Bloque 4 - Probar relaciones y transacciones	55
T4.1 - Probar relaciones @ManyToOne y @OneToMany	55
T4.2 - Probar FetchType.LAZY	56

T4.3 - Probar transacciones y rollback	57
Bloque 5 - Buenas prácticas y errores	57
T5.1 - Buenas prácticas en tests de JPA	57
T5.2 - Errores comunes	58
T5.3 - Cuándo usar @DataJpaTest y cuándo @SpringBootTest	59
Resumen de la teoría	59
Criterio profesional añadido	59

Módulo 4 - Persistencia de datos con JPA / Hibernate

Baseline: Java 17, Spring Boot 3.5.16 y Maven Wrapper 3.9.16. M4 evoluciona el proyecto acumulativo de M3: mantiene el contrato REST y sustituye la persistencia en memoria por JPA/Hibernate.

Punto 4.1 - Introducción a JPA e Hibernate

Objetivos de aprendizaje

1. Explicar qué es JPA, qué es Hibernate y qué es Spring Data JPA, y cómo se relacionan.
2. Entender por qué necesitamos una base de datos real en lugar de un repositorio en memoria.
3. Configurar Spring Boot para conectarse a una base de datos relacional.
4. Definir una entidad JPA con anotaciones básicas.
5. Crear un repositorio de Spring Data JPA.
6. Verificar que la aplicación persiste datos entre reinicios.
7. Diagnosticar y resolver los errores más comunes al configurar JPA.

Bloque 1 - El problema de la persistencia

T1.1 - Qué significa persistir datos

Hasta ahora, nuestros repositorios han sido en memoria: guardaban los datos en un Map o en una lista. Eso significa que los datos viven mientras la aplicación está arrancada. Si paras la aplicación, los datos desaparecen. Si la reinicias, vuelven a aparecer los datos iniciales del CommandLineRunner, pero los que hubieras creado durante la ejecución se pierden.

Persistir datos significa guardarlos en un medio que sobreviva al proceso: un fichero, una base de datos, un servicio externo. Cuando un ciudadano crea una solicitud de beca en el sistema del Ministerio, esa solicitud debe seguir ahí mañana, cuando el funcionario la revise. No puede desaparecer al reiniciar la aplicación.

En una aplicación real, la persistencia se hace con una base de datos. Una base de datos es un sistema que almacena datos de forma estructurada, permite consultarlos eficientemente y garantiza que no se corrompan. Hay dos grandes familias:

- Bases de datos relacionales: los datos se organizan en tablas con filas y columnas. Usan SQL como lenguaje de consulta. Ejemplos: PostgreSQL, MySQL, Oracle, SQL Server, H2.
- Bases de datos no relacionales (NoSQL): los datos se organizan de otras formas (documentos, grafos, clave-valor). Ejemplos: MongoDB, Redis, Cassandra.

En este curso usaremos una base de datos relacional, que es la más común en administraciones públicas. Y usaremos H2 en desarrollo, porque es una base de datos en memoria que no requiere instalación, y PostgreSQL en producción, porque es robusta y gratuita.

T1.2 - Qué es JPA

JPA son las siglas de Java Persistence API. Es una especificación de Java que define cómo mapear objetos Java a tablas de bases de datos relacionales. No es una librería ni un framework: es un

conjunto de interfaces y anotaciones que dicen cómo debe hacerse la persistencia.

JPA define:

- Anotaciones para marcar clases como entidades (@Entity), campos como columnas (@Column), relaciones entre entidades (@OneToMany, @ManyToOne), etc.
- Interfaces para realizar operaciones de persistencia (EntityManager, EntityManagerFactory).
- Un lenguaje de consulta llamado JPQL (Java Persistence Query Language), que es similar a SQL pero trabaja con objetos en lugar de tablas.

JPA no implementa nada: solo define el contrato. Para usarla, necesitas una implementación. La más usada es Hibernate.

T1.3 - Qué es Hibernate y qué es Spring Data JPA

Hibernate es la implementación de referencia de JPA. Es una librería que traduce las operaciones JPA a SQL, ejecuta las consultas contra la base de datos y mapea los resultados a objetos Java. Cuando usas JPA, por debajo está Hibernate haciendo el trabajo.

Spring Data JPA es una capa sobre JPA que simplifica aún más el acceso a datos. Proporciona:

- Repositorios automáticos: defines una interfaz que extiende JpaRepository y Spring genera la implementación. No tienes que escribir el código de acceso a datos.
- Consultas derivadas: defines métodos como findByDni y Spring genera la consulta SQL automáticamente.
- Paginación y ordenación: incluidas por defecto.
- Integración con Spring: transacciones, inyección de dependencias, etc.

La relación entre las tres es:

Spring Data JPA → JPA (especificación) → Hibernate (implementación)

Spring Data JPA usa JPA, y JPA usa Hibernate. Tú escribes código con Spring Data JPA, que es lo más cómodo. Por debajo, Hibernate hace el trabajo sucio.

Pregunta de reflexión: ¿Por qué crees que Spring Data JPA es útil si ya existe JPA? ¿Qué aporta?

Bloque 2 - Configuración de Spring Boot para JPA

T2.1 - La dependencia spring-boot-starter-data-jpa

Para usar JPA en Spring Boot, se añade la dependencia spring-boot-starter-data-jpa al pom.xml:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

Este starter trae: - Hibernate como implementación de JPA. - Spring Data JPA para los repositorios automáticos. - HikariCP como pool de conexiones. - Spring ORM para la integración con Spring.

También hay que añadir el driver de la base de datos. Para H2:

```
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
```

Para PostgreSQL:

```
<dependency>
  <groupId>org.postgresql</groupId>
```

```
<artifactId>postgresql</artifactId>
<scope>runtime</scope>
</dependency>
```

El `<scope>runtime</scope>` indica que la dependencia es necesaria en tiempo de ejecución, pero no en tiempo de compilación. No escribimos código que use directamente las clases del driver, así que no hace falta en compilación.

T2.2 - Configuración en application.properties

Spring Boot configura JPA automáticamente si detecta las dependencias. Pero hay que indicarle a qué base de datos conectarse. En `application.properties`:

Para H2 (desarrollo):

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
spring.h2.console.enabled=true
```

- `spring.datasource.url`: la URL de conexión. `jdbc:h2:mem:testdb` significa “base de datos H2 en memoria llamada testdb”.
- `spring.datasource.driver-class-name`: el driver JDBC.
- `spring.datasource.username` y `password`: credenciales. H2 por defecto usa `sa` sin contraseña.
- `spring.jpa.database-platform`: el dialecto de Hibernate para esa base de datos. Hibernate genera SQL específico para cada base de datos.
- `spring.jpa.hibernate.ddl-auto`: qué hacer con el esquema al arrancar. `create-drop` crea las tablas al arrancar y las borra al parar. `update` las actualiza sin borrar datos. `none` no hace nada.
- `spring.jpa.show-sql`: muestra el SQL que Hibernate ejecuta en los logs. Útil para depurar.
- `spring.h2.console.enabled`: habilita la consola web de H2 en `/h2-console`.

Para PostgreSQL (producción):

```
spring.datasource.url=jdbc:postgresql://localhost:5432/mibd
spring.datasource.username=usuario
spring.datasource.password=contraseña
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
```

En producción, `ddl-auto` suele ser `update` o `none`. `create-drop` borraría los datos al parar, lo cual es catastrófico en producción.

T2.3 - Perfiles para desarrollo y producción

Es buena práctica tener configuraciones distintas para desarrollo y producción. Se usan perfiles: `application.properties` (común):

```
spring.application.name=mi-proyecto
spring.profiles.active=dev
```

`application-dev.properties` (desarrollo):

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
```

```

spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
spring.h2.console.enabled=true

application-prod.properties (producción):

spring.datasource.url=jdbc:postgresql://localhost:5432/mibd
spring.datasource.username=${DB_USER}
spring.datasource.password=${DB_PASSWORD}
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false

```

En producción, las credenciales se leen de variables de entorno (\${DB_USER}, \${DB_PASSWORD}), no se escriben en el archivo. Eso evita que las credenciales se filtren en el repositorio.

Pregunta de reflexión: ¿Por qué es mala idea usar create-drop en producción? ¿Qué pasaría con los datos?

Bloque 3 - Entidades JPA

T3.1 - Qué es una entidad

Una entidad es una clase Java que representa una tabla de la base de datos. Cada instancia de la entidad representa una fila de esa tabla. Cada campo de la entidad representa una columna.

Por ejemplo, una entidad Alumno representa la tabla alumnos, con columnas id, nombre, apellidos, dni, fecha_nacimiento, curso. Cada instancia de Alumno es una fila con esos valores.

Las entidades se marcan con la anotación @Entity. Esa anotación le dice a JPA que la clase debe mapearse a una tabla.

```

@Entity
@Table(name = "alumnos")
public class Alumno {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String nombre;

    // ...
}

```

T3.2 - Anotaciones básicas

Las anotaciones más usadas en una entidad son:

- @Entity: marca la clase como entidad. Obligatoria.
- @Table(name = "..."): especifica el nombre de la tabla. Opcional; si no se pone, se usa el nombre de la clase en minúsculas.
- @Id: marca el campo que es la clave primaria. Obligatoria.
- @GeneratedValue: indica que el valor del ID se genera automáticamente. Las estrategias más comunes son:
 - IDENTITY: la base de datos genera el ID (autoincrement).

- SEQUENCE: se usa una secuencia de la base de datos.
- AUTO: Hibernate elige la estrategia según la base de datos.
- @Column: personaliza la columna. Atributos comunes:
 - name: nombre de la columna (si difiere del campo).
 - nullable: si acepta nulos.
 - length: longitud máxima (para strings).
 - unique: si debe ser único.
- @Enumerated: para campos que son enums. EnumType.STRING guarda el nombre del enum; EnumType.ORDINAL guarda el índice (menos recomendado).
- @Temporal: para fechas. En Java 8+ se usa LocalDate, LocalDateTime, que no necesitan @Temporal.

T3.3 - Ejemplo completo de entidad

```
package es.mecd.demo.miproyecto.alumno;

import jakarta.persistence.*;
import java.time.LocalDate;
@Entity
@Table(name = "alumnos")
public class Alumno {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String nombre;

    @Column(nullable = false, length = 150)
    private String apellidos;

    @Column(nullable = false, unique = true, length = 9)
    private String dni;

    @Column(name = "fecha_nacimiento")
    private LocalDate fechaNacimiento;

    @Column(nullable = false, length = 50)
    private String curso;

    public Alumno() {
    }

    public Alumno(String nombre, String apellidos, String dni,
        LocalDate fechaNacimiento, String curso) {
        this.nombre = nombre;
        this.apellidos = apellidos;
        this.dni = dni;
        this.fechaNacimiento = fechaNacimiento;
        this.curso = curso;
    }

    // getters y setters
}
```

```
}
```

Fíjate en varios detalles:

- El ID es Long y se genera automáticamente. No lo asigna el cliente; lo asigna la base de datos.
- @Column(nullable = false) indica que la columna no acepta nulos. Si se intenta guardar un alumno sin nombre, la base de datos lanza un error.
- @Column(unique = true) en el DNI indica que no puede haber dos alumnos con el mismo DNI. La base de datos lo garantiza.
- @Column(name = "fecha_nacimiento") mapea el campo fechaNacimiento a la columna fecha_nacimiento (snake_case, que es la convención en bases de datos).
- El constructor sin argumentos es obligatorio: JPA lo necesita para crear instancias al leer de la base de datos. > **Pregunta de reflexión:** ¿Por qué el ID se genera automáticamente en lugar de asignarlo el cliente? ¿Qué ventaja tiene?

Bloque 4 - Repositorios Spring Data JPA

T4.1 - La interfaz JpaRepository

Con Spring Data JPA, el repositorio es una interfaz que extiende JpaRepository:

```
package es.mecd.demo.miproyecto.alumno;

import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface AlumnoRepository extends JpaRepository<Alumno, Long> {

    Optional<Alumno> findByDni(String dni);

    boolean existsByDni(String dni);
}
```

JpaRepository<Alumno, Long> indica que el repositorio gestiona entidades Alumno con ID de tipo Long. Spring genera la implementación automáticamente. No hay que escribir nada.

Métodos heredados de JpaRepository:

- findAll(): devuelve todas las entidades.
- findById(Long id): devuelve una entidad por ID (Optional).
- save(Alumno alumno): inserta o actualiza.
- deleteById(Long id): elimina por ID.
- count(): devuelve el número de entidades.
- existsById(Long id): comprueba si existe.

Métodos derivados: Spring Data JPA genera la consulta a partir del nombre del método. Por ejemplo:

- findByDni(String dni) → SELECT * FROM alumnos WHERE dni = ?.
- findByCurso(String curso) → SELECT * FROM alumnos WHERE curso = ?.
- findByNombreAndApellidos(String nombre, String apellidos) → SELECT * FROM alumnos WHERE nombre = ? AND apellidos = ?.
- existsByDni(String dni) → SELECT COUNT(*) > 0 FROM alumnos WHERE dni = ?.

La convención de nombres es: findBy + nombre del campo + operador (And, Or, GreaterThan, LessThan, etc.).

T4.2 - Consultas personalizadas con @Query

Cuando el nombre del método no basta, se usa la anotación @Query con JPQL:

```

@Query(
    "SELECT a FROM Alumno a "
    + "WHERE a.curso = :curso AND a.fechaNacimiento > :fecha"
)
List<Alumno> buscarPorCursoYNacidosDespues(
    @Param("curso") String curso,
    @Param("fecha") LocalDate fecha);

```

JPQL es similar a SQL, pero trabaja con objetos en lugar de tablas. Alumno a significa “la entidad Alumno, alias a”. a.curso significa “el campo curso de la entidad”. No se usan nombres de tablas ni de columnas, sino nombres de clases y campos.

También se puede usar SQL nativo con nativeQuery = true:

```

@Query(value = "SELECT * FROM alumnos WHERE curso = :curso", nativeQuery = true)
List<Alumno> buscarPorCursoNativo(@Param("curso") String curso);

```

Pero JPQL es preferible porque es independiente de la base de datos.

T4.3 - Paginación y ordenación

Spring Data JPA soporta paginación y ordenación de forma nativa:

```
Page<Alumno> findByCurso(String curso, Pageable pageable);
```

Pageable es una interfaz que encapsula la página, el tamaño y la ordenación. Se construye con PageRequest:

```

Pageable pageable = PageRequest.of(0, 20, Sort.by("nombre").ascending());
Page<Alumno> pagina = repository.findByCurso("5º", pageable);

```

Page<Alumno> contiene los elementos de la página y metadatos: total de elementos, total de páginas, número de página, etc. Es lo que devuelve Spring Data JPA cuando se pagina.

Pregunta de reflexión: ¿Qué ventaja tiene que Spring Data JPA genere las consultas a partir del nombre del método? ¿Qué inconveniente tiene?

Bloque 5 - Transacciones y buenas prácticas

T5.1 - Qué es una transacción

Una transacción es un conjunto de operaciones que se ejecutan como una unidad atómica: o se ejecutan todas, o no se ejecuta ninguna. Si una falla, se hace rollback (se deshacen los cambios). Si todas tienen éxito, se hace commit (se confirman los cambios).

En Spring, las transacciones se gestionan con la anotación @Transactional. Se pone en el servicio, no en el repositorio:

```

@Service
public class AlumnoService {

    @Transactional
    public AlumnoDTO crear(AlumnoRequestDTO request) {
        // ...
    }
}

```

@Transactional envuelve el método en una transacción. Si el método lanza una excepción que activa rollback, Spring deshace la unidad de trabajo; si termina normalmente, hace commit.

Para consultas, se puede usar @Transactional(readOnly = true), que optimiza el rendimiento porque Hibernate no tiene que comprobar cambios.

T5.2 - Buenas prácticas con JPA

Usar DTOs, no entidades, en la API. Las entidades no deben exponerse directamente. Se transforman a DTOs en el servicio.

No usar EAGER por defecto. Las relaciones @OneToMany y @ManyToOne se cargan de forma perezosa (LAZY) por defecto en JPA, pero Hibernate puede cambiarlo. Conviene revisar la configuración.

Evitar el problema N+1. Cuando se cargan relaciones perezosas en un bucle, se lanzan N+1 consultas (una por cada elemento). Se soluciona con JOIN FETCH en JPQL o con @EntityGraph.

Usar @Transactional en el servicio, no en el repositorio. El repositorio ya tiene transacciones por defecto en los métodos de JpaRepository. El servicio es el que decide la unidad de trabajo.

No usar ddl-auto=create-drop en producción. Borra los datos al parar. Usar update o none.

Versionar el esquema con Flyway o Liquibase. En producción, el esquema debe gestionarse con migraciones versionadas, no con ddl-auto.

T5.3 - El problema N+1

El problema N+1 ocurre cuando se carga una lista de entidades y, por cada una, se lanza una consulta adicional para cargar una relación. Por ejemplo, si cargas 100 alumnos y cada uno tiene un curso, se lanzan 1 consulta para los alumnos y 100 para los cursos. Total: 101 consultas.

Se detecta activando spring.jpa.show-sql=true y observando los logs. Si ves muchas consultas repetidas, tienes un N+1.

Se soluciona con:

- JOIN FETCH en JPQL: SELECT a FROM Alumno a JOIN FETCH a.curso.
- @EntityGraph: una anotación que indica qué relaciones cargar de forma ansiosa.
- @BatchSize: carga las relaciones en lotes.

En este curso, como las relaciones las veremos en el punto 4.4, dejaremos el N+1 para entonces.

Pregunta de reflexión: ¿Por qué el problema N+1 es peligroso en producción? ¿Qué impacto tiene en el rendimiento?

Resumen de la teoría

- Persistir datos significa guardarlos en un medio que sobreviva al proceso.
- JPA es una especificación de Java para mapear objetos a tablas.
- Hibernate es la implementación de referencia de JPA.
- Spring Data JPA es una capa sobre JPA que simplifica el acceso a datos.
- Dependencias: spring-boot-starter-data-jpa + driver de la base de datos.
- Configuración: application.properties con URL, credenciales y dialecto.
- Perfiles: dev con H2, prod con PostgreSQL.
- Entidad: clase con @Entity, @Id, @Column.
- Repositorio: interfaz que extiende JpaRepository.
- Consultas derivadas: findByDni, existsByDni.
- @Query: para consultas personalizadas en JPQL.
- Paginación: Pageable y Page.
- Transacciones: @Transactional en el servicio.
- Buenas prácticas: DTOs, LAZY, evitar N+1, no usar create-drop en producción.

Criterio profesional añadido

En Spring Boot 3.x la persistencia usa el espacio de nombres jakarta.persistence. La transacción pertenece a la capa de servicio, las entidades no deben ser el contrato HTTP y cualquier consulta potencialmente costosa debe comprobarse con SQL/logs y pruebas de repositorio.

Punto 4.2 - Definición de entidades

Objetivos de aprendizaje

1. Explicar el ciclo de vida de una entidad JPA y sus estados.
2. Elegir la estrategia de generación de ID adecuada para cada caso.
3. Configurar correctamente los tipos de datos de las columnas.
4. Usar anotaciones avanzadas: @Enumerated, @Lob, @Transient, @Embedded.
5. Implementar equals y hashCode correctamente en entidades JPA.
6. Diseñar entidades siguiendo buenas prácticas.
7. Diagnosticar y resolver los errores más comunes al definir entidades.

Bloque 1 - Qué es una entidad y su ciclo de vida

T1.1 - El ciclo de vida de una entidad

Una entidad JPA no es una clase Java normal. Es una clase cuyo ciclo de vida está gestionado por el EntityManager de JPA. A lo largo de su vida, una entidad pasa por varios estados:

Transient (nueva):

- La entidad se ha creado con new, pero no está asociada a ninguna sesión de persistencia.
- No tiene representación en la base de datos.
- No está gestionada por el EntityManager.
- Si se modifica, no se refleja en la base de datos.

Managed (gestionada):

- La entidad está asociada a una sesión de persistencia (normalmente dentro de una transacción).
- Tiene representación en la base de datos.
- El EntityManager la vigila: si se modifica, Hibernate genera el UPDATE correspondiente al hacer commit (dirty checking).
- Es el estado en el que están las entidades devueltas por un findById o save.

Detached (separada):

- La entidad estaba gestionada, pero la sesión se ha cerrado (por ejemplo, al terminar la transacción).
- Sigue teniendo representación en la base de datos, pero ya no está vigilada.
- Si se modifica, no se refleja en la base de datos hasta que se vuelve a asociar (merge).

Removed (eliminada):

- La entidad está marcada para eliminación.
- Al hacer commit, Hibernate genera el DELETE.

Entender estos estados es fundamental para trabajar con JPA. La mayoría de los errores vienen de no saber en qué estado está una entidad.

T1.2 - Requisitos de una clase entidad

Para que una clase sea una entidad JPA válida, debe cumplir:

1. Estar anotada con @Entity. Sin esta anotación, JPA no la reconoce.
2. Tener un campo anotado con @Id. Es la clave primaria. Sin ella, JPA no sabe cómo identificar las filas.
3. Tener un constructor sin argumentos (público o protegido). JPA lo usa para crear instancias al leer de la base de datos.
4. No ser final. JPA genera proxies para las entidades; si la clase es final, no puede.
5. Los campos no deben ser final. JPA los modifica al leer de la base de datos.
6. La clase debe ser de nivel superior. No puede ser una clase interna anónima.
7. Implementar Serializable (recomendado). No es obligatorio, pero es útil si la entidad se transmite entre capas o se cachea.

Además, JPA recomienda:

- No usar tipos primitivos para campos que pueden ser nulos. Usar Long en lugar de long, Integer en lugar de int. Así se distingue “valor 0” de “valor ausente”.
- Definir getters y setters. JPA los usa para acceder a los campos.
- Sobrescribir equals y hashCode. Para que las entidades se comporten correctamente en colecciones.

T1.3 - Entidad vs DTO: la separación definitiva

Ya hemos hablado de la diferencia entre entidad y DTO. En este punto, con JPA en juego, la separación es más importante que nunca.

La entidad representa una tabla de la base de datos. Sus campos coinciden con las columnas. Tiene anotaciones JPA. Su ciclo de vida lo gestiona el EntityManager. No debe exponerse en la API.

El DTO representa los datos que viajan entre el cliente y el servidor. No tiene anotaciones JPA. Es un simple contenedor de datos. Es lo que se expone en la API.

¿Por qué no exponer la entidad directamente?

- Seguridad. La entidad puede tener campos sensibles (contraseñas, tokens, auditoría) que no deben salir.
- Acoplamiento. Si expones la entidad, el contrato de la API queda atado a la estructura de la base de datos. Cualquier cambio en la tabla rompe la API.
- Serialización. Las entidades tienen relaciones que pueden provocar recursión infinita en JSON.
- Lazy loading. Las relaciones LAZY no se pueden serializar fuera de una transacción, lo que provoca LazyInitializationException.

La regla es: entidad para la base de datos, DTO para la API. El servicio transforma entre ambas.

Pregunta de reflexión: ¿Qué pasaría si expones una entidad con una relación @OneToMany LAZY directamente en un controlador? ¿Qué error verías?

Bloque 2 - Generación de identificadores

T2.1 - La anotación @Id

La anotación @Id marca el campo que es la clave primaria de la entidad. Toda entidad debe tener una. Sin ella, JPA no sabe cómo identificar cada fila.

El campo @Id puede ser:

- Un tipo primitivo o wrapper: Long, Integer, String, UUID.
- Un tipo compuesto: una clase con varios campos (@EmbeddedId o @IdClass). Poco común.

La elección del tipo de ID depende del caso:

- Long con autoincrement: lo más común. Simple, eficiente, ordenable.
- UUID: útil cuando los IDs se generan en el cliente o cuando no quieres exponer el número de registros. Ocupa más espacio.
- String: para IDs con significado (códigos de expediente, por ejemplo). Requiere que el cliente o el servidor lo generen manualmente.

En este curso usaremos Long con autoincrement, que es lo más habitual.

T2.2 - Estrategias de generación

La anotación @GeneratedValue indica que el valor del ID se genera automáticamente. Tiene varias estrategias:

GenerationType.IDENTITY:

- La base de datos genera el ID con una columna autoincrement.
- Hibernate no interviene en la generación.

- Ventaja: simple, funciona en MySQL, PostgreSQL, H2, SQL Server.
- Inconveniente: no permite generar el ID antes del INSERT; Hibernate tiene que ejecutar el INSERT para obtener el ID.
- Uso típico: la mayoría de los casos.

GenerationType.SEQUENCE:

- Se usa una secuencia de la base de datos.
- Hibernate pide el siguiente valor de la secuencia antes del INSERT.
- Ventaja: permite el batch insert (varios INSERT en uno).
- Inconveniente: requiere que la base de datos soporte secuencias (PostgreSQL, Oracle, H2; MySQL no).
- Uso típico: cuando se necesita rendimiento en inserciones masivas. GenerationType.AUTO:
- Hibernate elige la estrategia según la base de datos.
- Ventaja: portabilidad.
- Inconveniente: menos control.
- Uso típico: cuando no quieres atarte a una base de datos.

GenerationType.TABLE:

- Se usa una tabla específica para generar IDs.
- Ventaja: portabilidad total.
- Inconveniente: lento, no recomendado.
- Uso típico: casi nunca.

La recomendación es IDENTITY para la mayoría de los casos, y SEQUENCE cuando se necesita rendimiento en inserciones masivas.

T2.3 - IDs en la API

Un detalle importante: aunque el ID sea Long en la entidad, en la API puede exponerse como String. Eso evita que los clientes hagan suposiciones sobre el rango de IDs (por ejemplo, que son consecutivos) y facilita la migración a otros tipos de ID en el futuro. En el DTO de salida:

```
@JsonProperty("id")
private String identificador;
```

Y en el servicio, se convierte:

```
dto.setIdentificador(entidad.getId() != null ? entidad.getId().toString() : null);
```

En el controlador, el @PathVariable sigue siendo Long porque la URL lleva el número:

```
@GetMapping("/{id}")
public ResponseEntity<AlumnoDTO> consultar(@PathVariable Long id) {
    // ...
}
```

Esta separación entre el tipo del ID en la entidad (Long) y en la API (String) es una buena práctica.

Pregunta de reflexión: ¿Por qué es buena idea que el ID se exponga como String en la API aunque en la base de datos sea Long?

Bloque 3 - Mapeo de columnas

T3.1 - La anotación @Column

La anotación @Column personaliza el mapeo de un campo a una columna. Sus atributos más importantes son:

- name: el nombre de la columna en la base de datos. Si no se pone, se usa el nombre del campo.
- nullable: si la columna acepta valores nulos. Por defecto true.
- unique: si la columna debe tener valores únicos. Por defecto false.
- length: la longitud máxima para strings. Por defecto 255.
- precision y scale: para números decimales. precision es el total de dígitos; scale, los decimales.
- insertable y updatable: si la columna se incluye en los INSERT y UPDATE. Útil para campos generados por la base de datos o de solo lectura.
- columnDefinition: el tipo SQL exacto. Se usa cuando se necesita algo que JPA no cubre.

Ejemplo:

```
@Column(name = "fecha_nacimiento", nullable = false)
private LocalDate fechaNacimiento;

@Column(nullable = false, unique = true, length = 9)
private String dni;
@Column(precision = 10, scale = 2)
private BigDecimal importe;
```

Convención de nombres: en Java se usa camelCase (fechaNacimiento); en la base de datos es habitual snake_case (fecha_nacimiento). Spring Boot aplica una estrategia física de nombres que normalmente realiza esta conversión; si se personaliza, debe hacerse de forma explícita y consistente.

T3.2 - Tipos de datos Java a SQL

JPA mapea los tipos Java a tipos SQL automáticamente. La tabla de conversión más común es:

Tipo Java Tipo SQL

String VARCHAR(n)

Integer / int INTEGER

Long / long BIGINT Tipo Java Tipo SQL

Double / double DOUBLE

Float / float FLOAT

BigDecimal DECIMAL(p,s)

Boolean / boolean BOOLEAN

LocalDate DATE

LocalTime TIME

LocalDateTime TIMESTAMP

byte[] BLOB

enum VARCHAR o INTEGER (según @Enumerated)

Para los decimales, usa BigDecimal, no Double. Double tiene problemas de precisión (0.1 + 0.2 no es exactamente 0.3). Para importes de dinero, BigDecimal es obligatorio. Para las fechas, usa las clases de Java 8: LocalDate, LocalTime, LocalDateTime. Son inmutables, no tienen problemas de zona horaria, y Hibernate las soporta de forma nativa.

T3.3 - Tipos especiales

@Enumerated: para campos que son enums. Tiene dos opciones:

- EnumType.STRING: guarda el nombre del enum ("EN_TRAMITE"). Recomendado.

- EnumType.ORDINAL: guarda el índice (0, 1, 2). No recomendado porque si se reordena el enum, los datos se corrompen.

```
@Enumerated(EnumType.STRING)
@Column(nullable = false, length = 20)
private EstadoExpediente estado;
```

@Lob se usa para campos grandes:

- @Lob con String: mapea a CLOB (texto grande). Útil para descripciones largas.
- @Lob con byte[]: mapea a BLOB (binario grande). Útil para ficheros, imágenes.

```
@Lob
@Column(name = "descripcion")
private String descripcion;
```

```
@Lob
@Column(name = "fichero")
private byte[] fichero;
```

@Temporal se usa con java.util.Date y java.util.Calendar. No se aplica a LocalDate ni LocalDateTime, que JPA mapea directamente.

```
@Temporal(TemporalType.DATE)
private Date fecha;
```

Pregunta de reflexión: ¿Por qué se recomienda EnumType.STRING en lugar de EnumType.ORDINAL? ¿Qué riesgo se evita?

Bloque 4 - Anotaciones avanzadas

T4.1 - @Transient

La anotación @Transient marca un campo que no se persiste en la base de datos. El campo existe en la clase Java, pero JPA lo ignora al generar el SQL. Se usa para:

- Campos calculados. Por ejemplo, un campo edad que se calcula a partir de fechaNacimiento.
- Campos temporales. Por ejemplo, un flag que solo se usa en memoria.
- Campos de conveniencia. Por ejemplo, un campo que agrupa otros campos para mostrarlos.

```
@Transient
private Integer edad;

public Integer getEdad() {
    if (fechaNacimiento == null) return null;
    return Period.between(fechaNacimiento, LocalDate.now()).getYears();
}
```

El campo edad no se guarda en la base de datos, pero se puede consultar desde el código.

Importante: no confundir @Transient de JPA (jakarta.persistence.Transient) con transient de Java (la palabra clave). Son cosas distintas. La primera es para JPA; la segunda, para la serialización de Java.

T4.2 - @Embedded y @Embeddable

A veces, varios campos de una entidad forman un grupo lógico que se repite en otras entidades. Por ejemplo, una dirección (calle, número, ciudad, código postal) puede aparecer en alumnos, centros, proveedores.

En lugar de repetir los campos en cada entidad, se puede crear una clase @Embeddable:

```
@Embeddable
public class Direccion {

    @Column(name = "calle")
    private String calle;

    @Column(name = "numero")
    private String numero;

    @Column(name = "ciudad")
    private String ciudad;

    @Column(name = "codigo_postal", length = 5)
    private String codigoPostal;

    // getters y setters
}
```

Y usarla en una entidad con @Embedded:

```
@Entity
@Table(name = "alumnos")
public class Alumno {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Embedded
    private Direccion direccion;

    // ...
}
```

En la base de datos, los campos de Direccion se mapean a columnas de la tabla alumnos. No se crea una tabla separada. Es una forma de agrupar campos sin crear una relación.

Ventaja: reutilización. La misma clase Direccion se puede usar en varias entidades.

Inconveniente: los campos se mezclan en la tabla. Si dos entidades tienen una Direccion, las columnas se llaman igual y puede haber conflictos. Se soluciona con @AttributeOverride.

T4.3 - @Version y bloqueo optimista

Cuando varias transacciones modifican la misma entidad a la vez, puede haber condiciones de carrera: una transacción sobrescribe los cambios de otra sin saberlo.

Para evitarlo, JPA ofrece el bloqueo optimista con la anotación @Version:

```
@Entity
@Table(name = "alumnos")
public class Alumno {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Version
    private Integer version;
```

```
// ...
}
```

Hibernate añade una columna version a la tabla. Cada vez que se actualiza la entidad, incrementa el número de versión. Si dos transacciones intentan actualizar a la vez: 1. La primera lee la entidad con version = 1. 2. La segunda también lee con version = 1. 3. La primera actualiza: UPDATE ... SET version = 2 WHERE id = ? AND version = 1. Éxito. 4. La segunda intenta actualizar: UPDATE ... SET version = 2 WHERE id = ? AND version = 1. Falla, porque la versión ya es 2. 5. Hibernate lanza OptimisticLockException.

El programador decide qué hacer: reintentar, mostrar un error al usuario, etc.

Ventaja: evita que una transacción sobrescriba los cambios de otra sin saberlo.

Inconveniente: hay que manejar la OptimisticLockException.

En la práctica, @Version se usa cuando hay riesgo de concurrencia. Para entidades que rara vez se modifican a la vez, no es imprescindible.

Pregunta de reflexión: ¿Qué diferencia hay entre bloqueo optimista y bloqueo pesimista? ¿Cuál usarías en una API REST?

Bloque 5 - Buenas prácticas y errores comunes

T5.1 - Buenas prácticas en el diseño de entidades

Usar DTOs, no entidades, en la API. Ya lo hemos dicho, pero es la regla más importante.

Usar tipos wrapper, no primitivos. Long en lugar de long, Integer en lugar de int. Así se distingue “valor 0” de “valor ausente”.

Usar BigDecimal para importes. Nunca Double ni Float. La precisión es crítica en dinero.

Usar LocalDate, LocalTime, LocalDateTime. Son las clases modernas, inmutables, sin problemas de zona horaria.

Definir la longitud de los strings. @Column(length = 100). Por defecto son 255; si no se ajusta, se desperdicia espacio o se trunca.

Usar @Enumerated(EnumType.STRING). Nunca ORDINAL.

Sobrescribir equals y hashCode. Sin ellos, las entidades no se comportan bien en colecciones. Lo veremos en el siguiente punto.

No usar relaciones EAGER por defecto. Las relaciones @OneToMany y @ManyToOne deben ser LAZY salvo que haya una razón.

No exponer la entidad en la API. Siempre transformar a DTO.

Documentar la entidad. Un Javadoc breve explicando qué representa.

T5.2 - equals y hashCode en entidades JPA

Sobrescribir equals y hashCode en entidades JPA es más delicado de lo que parece. Hay varias estrategias:

Estrategia 1 - Basado en el ID:

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Alumno)) return false;
    Alumno otro = (Alumno) o;
    return id != null && id.equals(otro.id);
}
```

```
@Override
public int hashCode() {
    return getClass().hashCode();
}
```

Ventaja: simple. Inconveniente: si el ID es null (entidad no persistida), dos entidades distintas se consideran iguales.

Estrategia 2 - Basado en un campo de negocio único:

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Alumno)) return false;
    Alumno otro = (Alumno) o;
    return dni != null && dni.equals(otro.dni);
}
```

```
@Override
public int hashCode() {
    return dni != null ? dni.hashCode() : 0;
}
```

Ventaja: funciona aunque el ID sea null. Inconveniente: requiere que el campo de negocio sea realmente único y no cambie.

Estrategia 3 - La recomendada por Vlad Mihalcea (experto en JPA):

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Alumno)) return false;
    Alumno otro = (Alumno) o;
    return id != null && id.equals(otro.id);
}
```

```
@Override
public int hashCode() {
    return getClass().hashCode();
}
```

El hashCode devuelve siempre el mismo valor para la clase. Eso significa que todas las entidades de la misma clase van al mismo cubo en un HashSet, lo que es menos eficiente, pero garantiza que el hashCode no cambia cuando el ID se asigna. Es un compromiso entre corrección y eficiencia.

En la práctica, para nuestro curso, usaremos la estrategia 3 o la 1. La estrategia 2 solo si hay un campo de negocio único claro.

T5.3 - Errores comunes al definir entidades

Error 1: olvidar el constructor sin argumentos.

JPA lo necesita para crear instancias al leer de la base de datos. Si no está, Hibernate lanza No default constructor for entity.

Error 2: usar final en la clase o en los campos.

JPA genera proxies para las entidades y necesita modificar los campos. Si son final, no puede.

Error 3: usar equals y hashCode basados en todos los campos.

Si un campo cambia, la entidad cambia de hashCode y se pierde en un HashSet. Usar solo el ID o un campo de negocio estable. Error 4: no definir length en los strings.

Por defecto 255. Si el campo es más largo, se trunca. Si es más corto, se desperdicia espacio.

Error 5: usar EnumType.ORDINAL.

Si se reordena el enum, los datos se corrompen.

Error 6: usar Double para importes.

Problemas de precisión. Usar BigDecimal.

Error 7: olvidar @Column(nullable = false) en campos obligatorios.

La base de datos acepta nulos. Si el código no los valida, se guardan datos incompletos.

Error 8: no usar @Version cuando hay concurrencia.

Dos transacciones pueden sobrescribirse sin saberlo.

Error 9: exponer la entidad en la API.

Acoplamiento, recursión, LazyInitializationException.

Error 10: usar @Data de Lombok en entidades. Lombok genera equals y hashCode basados en todos los campos, lo cual es problemático en JPA. Mejor usar @Getter y @Setter por separado, o escribir equals y hashCode manualmente.

Pregunta de reflexión: ¿Por qué @Data de Lombok es problemático en entidades JPA?
¿Qué alternativas hay?

Resumen de la teoría

- Ciclo de vida de una entidad: transient, managed, detached, removed.
- Requisitos: @Entity, @Id, constructor sin argumentos, no final.
- Generación de IDs: IDENTITY (recomendado), SEQUENCE (batch insert), AUTO, TABLE.
- @Column: name, nullable, unique, length, precision, scale.
- Tipos de datos: BigDecimal para importes, LocalDate para fechas, EnumType.STRING para enums.
- @Enumerated, @Lob, @Temporal.
- @Transient: campos no persistidos.
- @Embedded y @Embeddable: grupos de campos reutilizables.
- @Version: bloqueo optimista.
- Buenas prácticas: DTOs, wrapper types, BigDecimal, LocalDate, @Enumerated(STRING).
- equals y hashCode: basados en el ID o en un campo de negocio único.
- Errores comunes: constructor, final, EnumType.ORDINAL, Double, @Data.

Criterio profesional añadido

En Spring Boot 3.x la persistencia usa el espacio de nombres `jakarta.persistence`. La transacción pertenece a la capa de servicio, las entidades no deben ser el contrato HTTP y cualquier consulta potencialmente costosa debe comprobarse con SQL/logs y pruebas de repositorio.

Punto 4.3 - Repositorios JPA

Objetivos de aprendizaje

1. Explicar la jerarquía de interfaces de Spring Data JPA.
2. Elegir la interfaz adecuada según las necesidades del proyecto.
3. Escribir métodos derivados siguiendo las convenciones de nombres.
4. Usar @Query para consultas personalizadas en JPQL y SQL nativo.

5. Implementar paginación y ordenación con Pageable, Page y Sort.
6. Crear consultas dinámicas con Specifications.
7. Diagnosticar y resolver los errores más comunes al definir repositorios.

Bloque 1 - La jerarquía de repositorios

T1.1 - Por qué hay varias interfaces

Spring Data JPA no ofrece una sola interfaz de repositorio, sino varias, organizadas en una jerarquía. Cada una añade funcionalidad sobre la anterior. La razón es permitir que elijas la que necesitas: si solo quieres operaciones CRUD básicas, usas una interfaz ligera; si necesitas paginación y ordenación, usas otra más completa.

La jerarquía es:

```
Repository<T, ID>
    ↑
CrudRepository<T, ID>
    ↑
PagingAndSortingRepository<T, ID>
    ↑
JpaRepository<T, ID>
```

Cada interfaz hereda los métodos de la anterior y añade los suyos. JpaRepository es la más completa; Repository es la más básica.

En la práctica, casi siempre se usa JpaRepository, porque incluye todo lo que suelen necesitar las aplicaciones. Pero conviene conocer las demás para saber qué se está heredando.

T1.2 - Repository y CrudRepository

Repository<T, ID> es la interfaz raíz. No declara ningún método. Es una interfaz marcadora: solo sirve para indicar a Spring Data que la interfaz es un repositorio. Se usa cuando quieres declarar tus propios métodos sin heredar ninguno.

```
public interface AlumnoRepository extends Repository<Alumno, Long> {
    List<Alumno> findByCurso(String curso);
}
```

CrudRepository<T, ID> añade los métodos CRUD básicos:

- save(S entity): inserta o actualiza.
- saveAll(Iterable entities): inserta o actualiza varios.
- findById(ID id): busca por ID, devuelve Optional.
- findAll(): devuelve todos los elementos.
- findAllById(Iterable ids): devuelve los que coincidan con los IDs.
- count(): devuelve el número de elementos.
- deleteById(ID id): elimina por ID.
- delete(T entity): elimina la entidad.
- deleteAll(): elimina todos.
- existsById(ID id): comprueba si existe.

Es útil cuando no necesitas paginación ni ordenación. Para APIs simples, es suficiente.

T1.3 - PagingAndSortingRepository y JpaRepository

PagingAndSortingRepository<T, ID> añade a CrudRepository:

- findAll(Sort sort): devuelve todos los elementos ordenados.
- findAll(Pageable pageable): devuelve una página de elementos.

JpaRepository<T, ID> añade a PagingAndSortingRepository:

- `findAll()`: devuelve todos como List en lugar de Iterable.
- `findAll(Sort)`: devuelve como List.
- `findAll(Pageable)`: devuelve como Page.
- `saveAndFlush(S entity)`: guarda y fuerza el flush inmediato.
- `flush()`: fuerza el flush de todos los cambios pendientes.
- `deleteAllInBatch()`: elimina todos en una sola sentencia (no elemento por elemento).
- `deleteInBatch(Iterable entities)`: elimina varios en una sola sentencia.
- `getById(ID id)`: obtiene una referencia sin cargarla (lazy).
- `getOne(ID id)`: igual que `getById`.

Las diferencias prácticas entre `CrudRepository` y `JpaRepository` son:

- `JpaRepository` devuelve List en lugar de Iterable. Es más cómodo.
- `JpaRepository` tiene `flush()` y `saveAndFlush()`, útiles para tests.
- `JpaRepository` tiene `deleteAllInBatch()`, más eficiente para borrados masivos.

En la práctica, se usa `JpaRepository` por defecto, salvo que quieras restringir los métodos disponibles.

Pregunta de reflexión: ¿Qué diferencia hay entre `findAll()` en `CrudRepository` y `findAll()` en `JpaRepository`? ¿Cuál es más cómodo?

Bloque 2 - Métodos derivados

T2.1 - Qué son y cómo se generan

Los métodos derivados son métodos cuyo nombre sigue una convención y Spring Data JPA genera la consulta automáticamente a partir de él. No tienes que escribir SQL ni JPQL: solo declarar el método.

```
public interface AlumnoRepository extends JpaRepository<Alumno, Long> {

    Optional<Alumno> findByDni(String dni);

    List<Alumno> findByCurso(String curso);

    boolean existsByDni(String dni);

    long countByCurso(String curso);

    void deleteByDni(String dni);
}
```

Spring Data JPA analiza el nombre y genera la consulta:

- `findByDni(String dni)` → `SELECT a FROM Alumno a WHERE a.dni = ?.`
- `findByCurso(String curso)` → `SELECT a FROM Alumno a WHERE a.curso = ?.`
- `existsByDni(String dni)` → `SELECT COUNT(a) > 0 FROM Alumno a WHERE a.dni = ?.`
- `countByCurso(String curso)` → `SELECT COUNT(a) FROM Alumno a WHERE a.curso = ?.`
- `deleteByDni(String dni)` → `DELETE FROM Alumno a WHERE a.dni = ?.`

Los prefijos más comunes son: - `findBy`: devuelve una lista o un `Optional`. - `readBy`: igual que `findBy`. - `queryBy`: igual que `findBy`. - `getBy`: igual que `findBy`. - `countBy`: devuelve un `long`. - `existsBy`: devuelve un `boolean`. - `deleteBy`: elimina y devuelve `long` (número de elementos eliminados). - `removeBy`: igual que `deleteBy`.

T2.2 - Operadores y combinaciones

El nombre del método puede incluir operadores y combinaciones de campos:

Palabra clave Ejemplo Consulta generada

And findByNombreAndApellidos WHERE nombre = ? AND apellidos = ?

Or findByNombreOrApellidos WHERE nombre = ? OR apellidos = ? Palabra clave Ejemplo Consulta generada

Is findByNombreIs WHERE nombre = ?

Equals findByNombreEquals WHERE nombre = ?

Between findByFechaBetween WHERE fecha BETWEEN ? AND ?

LessThan findByEdadLessThan WHERE edad < ?

LessThanEqual findByEdadLessThanEqual WHERE edad <= ?

GreaterThan findByEdadGreaterThan WHERE edad > ?

GreaterThanEqual findByEdadGreaterThanEqual WHERE edad >= ?

Like findByNombreLike WHERE nombre LIKE ?

NotLike findByNombreNotLike WHERE nombre NOT LIKE ?

StartingWith findByNombreStartingWith WHERE nombre LIKE '??'

EndingWith findByNombreEndingWith WHERE nombre LIKE '??' Palabra clave Ejemplo Consulta generada

Containing findByNombreContaining WHERE nombre LIKE '%%?%%'

Not findByNombreNot WHERE nombre <> ?

In findByCursoIn WHERE curso IN (?, ?, ...)

NotIn findByCursoNotIn WHERE curso NOT IN (?, ?, ...)

IsNull findByFechaNacimientoIsNull WHERE fecha_nacimiento IS NULL

IsNotNull findByFechaNacimientoIsNotNull WHERE fecha_nacimiento IS NOT NULL

True findByActivoTrue WHERE activo = true

False findByActivoFalse WHERE activo = false

OrderBy findByCursoOrderByNombreAsc WHERE curso = ? ORDER BY nombre ASC

Ejemplos:

```
List<Alumno> findByCursoAndEstado(String curso, EstadoAlumno estado);
```

```
List<Alumno> findByFechaNacimientoBetween(LocalDate desde, LocalDate hasta);
```

```
List<Alumno> findByNombreContainingIgnoreCase(String texto);
```

```
List<Alumno> findByCursoOrderByApellidosAsc(String curso);
```

```
List<Alumno> findByCursoIn(List<String> cursos);
```

```
boolean existsByDniAndCurso(String dni, String curso);
```

IgnoreCase se puede añadir al final de un campo para ignorar mayúsculas: findByNombreIgnoreCase.

T2.3 - Limitaciones de los métodos derivados

Los métodos derivados son cómodos, pero tienen limitaciones:

- Nombres largos. Un método con tres o cuatro condiciones puede tener un nombre de 60 caracteres. Difícil de leer.

- Complejidad. No se pueden expresar consultas con subconsultas, joins complejos o funciones de agregación.
- Refactorización frágil. Si renombras un campo de la entidad, tienes que renombrar todos los métodos que lo usan. El IDE lo hace, pero es un cambio global.
- Errores al arrancar. Si el nombre del método no se puede interpretar, Spring Data falla al arrancar la aplicación. Es bueno (detecta el error pronto), pero puede bloquear el arranque.

La regla es: usa métodos derivados cuando la consulta es simple (una o dos condiciones). Cuando la consulta es compleja, usa @Query o Specifications.

Pregunta de reflexión: ¿Qué método derivado escribirías para buscar alumnos de un curso concreto ordenados por apellidos? ¿Y para buscar por nombre que contenga un texto, ignorando mayúsculas?

Bloque 3 - Consultas con @Query

T3.1 - JPQL con @Query

La anotación @Query permite escribir consultas personalizadas en JPQL (Java Persistence Query Language). JPQL es similar a SQL, pero trabaja con objetos en lugar de tablas.

```
@Query("SELECT a FROM Alumno a WHERE a.curso = :curso AND a.estado = :estado")
List<Alumno> buscarPorCursoYEstado(
    @Param("curso") String curso,
    @Param("estado") EstadoAlumno estado);
```

Alumno a significa “la entidad Alumno, con alias a”. Se usa el nombre de la clase, no el de la tabla. a.curso significa “el campo curso de la entidad”. Se usa el nombre del campo, no el de la columna. :curso y :estado son parámetros nombrados. Se referencian con @Param en los argumentos.

SELECT a devuelve la entidad completa. También se puede devolver solo algunos campos:

```
@Query("SELECT a.nombre, a.apellidos FROM Alumno a WHERE a.curso = :curso")
List<Object[]> buscarNombresPorCurso(@Param("curso") String curso);
```

Pero devolver Object[] es incómodo. En la práctica, se devuelven entidades o DTOs (con un constructor que acepte los campos).

T3.2 - Consultas de actualización y borrado

@Query también permite escribir consultas de actualización y borrado con @Modifying:

```
@Modifying
@Query("UPDATE Alumno a SET a.estado = :estado WHERE a.curso = :curso")
int actualizarEstadoPorCurso(
    @Param("curso") String curso,
    @Param("estado") EstadoAlumno estado);

@Modifying
@Query("DELETE FROM Alumno a WHERE a.estado = :estado")
int eliminarPorEstado(@Param("estado") EstadoAlumno estado);
```

@Modifying indica que la consulta cambia datos. Sin ella, Spring Data interpreta @Query como una consulta de lectura y la operación falla.

El valor de retorno es int (o void), con el número de filas afectadas.

Importante: las consultas @Modifying requieren una transacción. Se llaman desde un método del servicio anotado con @Transactional. Si no hay transacción, Spring lanza TransactionRequiredException.

Flush automático: por defecto, `@Modifying` no hace flush del contexto de persistencia antes de ejecutar la consulta. Si tienes cambios pendientes en el contexto, puede que la consulta no los vea. Para forzar el flush, se usa `@Modifying(flushAutomatically = true)`. Para limpiar el contexto después, `@Modifying(clearAutomatically = true)`.

T3.3 - SQL nativo con `@Query`

También se puede escribir SQL nativo con `nativeQuery = true`:

```
@Query(value = "SELECT * FROM alumnos WHERE curso = :curso", nativeQuery = true)
List<Alumno> buscarPorCursoNativo(@Param("curso") String curso);
```

Ventajas:

- Se puede usar cualquier función específica de la base de datos.
- Se pueden escribir consultas muy optimizadas.

Inconvenientes:

- No es portable: si cambias de base de datos, la consulta puede no funcionar.
- No usa el mapeo de JPA: los nombres de tablas y columnas son los reales.
- Los resultados se mapean a entidades por convención, pero puede haber problemas si los nombres no coinciden.

En la práctica, JPQL es preferible salvo que haya una razón de peso para usar SQL nativo (rendimiento, funciones específicas).

Pregunta de reflexión: ¿Qué diferencia hay entre JPQL y SQL nativo? ¿Cuándo usarías cada uno?

Bloque 4 - Paginación y ordenación

T4.1 - Pageable, Page y Sort

Spring Data JPA ofrece tres abstracciones para paginar y ordenar:

Sort representa el criterio de ordenación: qué campo y en qué dirección.

```
Sort sort = Sort.by("nombre").ascending();
Sort sortMultiple = Sort.by("curso")
    .ascending()
    .and(Sort.by("apellidos").descending());
```

Pageable encapsula la página, el tamaño y la ordenación.

```
Pageable pageable = PageRequest.of(0, 20, Sort.by("nombre").ascending());
```

- `PageRequest.of(page, size)`: página 0, tamaño 20, sin ordenación.
- `PageRequest.of(page, size, sort)`: con ordenación.
- `PageRequest.of(page, size, direction, properties...)`: con dirección y campos.

Page es el resultado de una consulta paginada. Contiene:

- Los elementos de la página actual: `getContent()`.
- El número de página actual: `getNumber()`.
- El tamaño de la página: `getSize()`.
- El total de elementos: `getTotalElements()`.
- El total de páginas: `getTotalPages()`.
- Si es la primera o la última página: `isFirst()`, `isLast()`.
- Si hay siguiente o anterior: `hasNext()`, `hasPrevious()`.

T4.2 - Cómo se usa en el repositorio

En el repositorio, los métodos que aceptan Pageable devuelven Page:

```
Page<Alumno> findByCurso(String curso, Pageable pageable);
```

```
Page<Alumno> findAll(Pageable pageable);
```

Si solo se quiere ordenar, sin paginar, se acepta Sort y se devuelve List:

```
List<Alumno> findByCurso(String curso, Sort sort);
```

```
List<Alumno> findAll(Sort sort);
```

Spring Data JPA genera la consulta paginada con LIMIT y OFFSET, y una consulta adicional de COUNT para obtener el total. En el servicio, se construye el Pageable a partir de los parámetros del controlador:

```
public Page<AlumnoDTO> listar(int page, int size, String sort) {
    Pageable pageable = PageRequest.of(page, size, Sort.by(sort).ascending());
    return repositorio.findAll(pageable).map(this::toDTO);
}
```

En el controlador, se aceptan los parámetros como query params:

```
@GetMapping
public Page<AlumnoDTO> listar(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "20") int size,
    @RequestParam(defaultValue = "id") String sort) {
    return service.listar(page, size, sort);
}
```

Y Spring serializa Page a JSON con los metadatos:

```
{
  "content": [...],
  "pageable": {...},
  "totalElements": 47,
  "totalPages": 3,
  "number": 0,
  "size": 20,
  "first": true,
  "last": false
}
```

T4.3 - Buenas prácticas con paginación

Limitar el tamaño máximo. No permitir size = 10000. Validar que size esté entre 1 y un máximo razonable (por ejemplo, 100).

Ordenación por defecto. Si el cliente no indica ordenación, usar una por defecto (normalmente por ID). Evita resultados inconsistentes entre páginas.

Usar Page en la API. El cliente necesita los metadatos para construir controles de navegación.

No paginar consultas muy pequeñas. Si sabes que una colección siempre tiene menos de 50 elementos, no hace falta paginar.

Documentar los parámetros. En OpenAPI, describir page, size y sort.

Pregunta de reflexión: ¿Por qué es importante limitar el tamaño máximo de página?
¿Qué problema evita?

Bloque 5 - Consultas dinámicas con Specifications

T5.1 - El problema de las consultas dinámicas

Imagina un endpoint de búsqueda que acepta varios filtros opcionales: curso, estado, rango de fechas, texto libre. El cliente puede enviar cualquier combinación. Con métodos derivados, tendrías que crear un método por cada combinación posible. Con @Query, tendrías que escribir múltiples consultas.

Las Specifications resuelven este problema: permiten construir consultas dinámicamente, combinando criterios según los filtros que lleguen.

Spring Data JPA ofrece JpaSpecificationExecutor para esto. El repositorio debe extender esta interfaz además de JpaRepository:

```
public interface AlumnoRepository extends JpaRepository<Alumno, Long>,
JpaSpecificationExecutor<Alumno> {
}
```

T5.2 - Cómo se construyen

Una Specification es una función que recibe un CriteriaBuilder, un CriteriaQuery y un Root, y devuelve un Predicate. En la práctica, se construye con lambdas:

```
public class AlumnoSpecifications {

    public static Specification<Alumno> porCurso(String curso) {
        return (root, query, cb) ->
            curso == null ? null : cb.equal(root.get("curso"), curso);
    }

    public static Specification<Alumno> porEstado(EstadoAlumno estado) {
        return (root, query, cb) ->
            estado == null ? null : cb.equal(root.get("estado"), estado);
    }

    public static Specification<Alumno> nacidoDespues(LocalDate fecha) {
        return (root, query, cb) ->
            fecha == null ? null : cb.greaterThan(root.get("fechaNacimiento"), fecha);
    }
}
```

Cada Specification devuelve un Predicate o null. Si devuelve null, se ignora. Para combinarlas:

```
Specification<Alumno> spec = Specification.where(porCurso(curso))
    .and(porEstado(estado))
    .and(nacidoDespues(fecha));
```

Y se pasa al repositorio:

```
List<Alumno> alumnos = repositorio.findAll(spec);
Page<Alumno> pagina = repositorio.findAll(spec, pageable);
```

Ventaja: las Specifications se combinan libremente. Si un filtro es null, no se aplica.

Inconveniente: más código y más complejo que los métodos derivados. Se usa cuando hay muchos filtros opcionales.

T5.3 - Cuándo usar cada enfoque

La elección entre métodos derivados, @Query y Specifications depende de la complejidad: Situación
Enfoque recomendado

Consulta simple con 1-2 condiciones Método derivado

Consulta con 3-4 condiciones fijas @Query

Consulta con joins complejos @Query

Consulta con agregaciones (COUNT, SUM, AVG) @Query

Consulta de actualización o borrado masivo @Query con @Modifying

Consulta dinámica con filtros opcionales Specifications

Necesitas paginación y ordenación Pageable + cualquiera

Regla: empieza con métodos derivados. Si el nombre se hace demasiado largo o la consulta es compleja, pasa a @Query. Si los filtros son dinámicos, usa Specifications.

Pregunta de reflexión: ¿Por qué las Specifications son útiles cuando hay filtros opcionales? ¿Qué problema evitan?

Resumen de la teoría

- Jerarquía: Repository → CrudRepository → PagingAndSortingRepository → JpaRepository.
- JpaRepository es la más usada: incluye CRUD, paginación, ordenación y batch.
- Métodos derivados: Spring genera la consulta a partir del nombre.
- Operadores: And, Or, Between, LessThan, Like, Containing, In, IsNull, OrderBy.
- @Query: JPQL o SQL nativo para consultas personalizadas.
- @Modifying: para consultas de actualización y borrado.
- Pageable, Page, Sort: paginación y ordenación.
- Specifications: consultas dinámicas con filtros opcionales.
- Buenas prácticas: limitar size, ordenación por defecto, documentar parámetros.

Criterio profesional añadido

En Spring Boot 3.x la persistencia usa el espacio de nombres `jakarta.persistence`. La transacción pertenece a la capa de servicio, las entidades no deben ser el contrato HTTP y cualquier consulta potencialmente costosa debe comprobarse con SQL/logs y pruebas de repositorio.

Punto 4.4 - Relaciones entre entidades: OneToMany, ManyToOne

Objetivos de aprendizaje

1. Explicar los cuatro tipos de relaciones JPA y cuándo usar cada uno.
2. Modelar una relación @ManyToOne y @OneToMany correctamente.
3. Entender el concepto de lado propietario y lado inverso de una relación.
4. Configurar cascade, fetch y orphanRemoval según el caso de uso.
5. Evitar el problema N+1 con JOIN FETCH y @EntityGraph.
6. Diagnosticar y resolver los errores más comunes al modelar relaciones.

Bloque 1 - Los cuatro tipos de relaciones

T1.1 - Cardinalidad de las relaciones

En una base de datos relacional, las tablas se relacionan entre sí con cardinalidades: uno a uno, uno a muchos, muchos a uno, muchos a muchos. JPA refleja esas cardinalidades con cuatro anotaciones:

- @OneToOne: un registro de A se relaciona con un único registro de B, y viceversa. Ejemplo: un alumno tiene una única ficha médica, y una ficha médica pertenece a un único alumno.

- @OneToMany: un registro de A se relaciona con varios registros de B. Ejemplo: un curso tiene muchos alumnos.
- @ManyToOne: varios registros de A se relacionan con un único registro de B. Es la cara inversa de @OneToMany. Ejemplo: muchos alumnos pertenecen a un único curso.
- @ManyToMany: varios registros de A se relacionan con varios registros de B. Ejemplo: un alumno se matricula en varias asignaturas, y una asignatura tiene varios alumnos.

En este curso nos centraremos en @OneToMany y @ManyToOne, que son las más comunes. Las otras dos las veremos de pasada.

T1.2 - Cómo se mapean a la base de datos

En una base de datos relacional, las relaciones se implementan con claves foráneas:

- @ManyToOne: se añade una columna en la tabla del “muchos” que apunta al ID de la tabla del “uno”. Por ejemplo, la tabla alumnos tiene una columna curso_id que apunta a la tabla cursos.
- @OneToMany: no se añade ninguna columna en la tabla del “uno”. La relación se ve desde el lado del “muchos”, usando la misma columna curso_id de la tabla alumnos.
- @OneToOne: se añade una columna en uno de los dos lados (normalmente en el que tiene la clave foránea) que apunta al ID del otro.
- @ManyToMany: se crea una tabla intermedia con dos columnas: una que apunta a A y otra que apunta a B. Por ejemplo, alumnos_asignaturas con alumno_id y asignatura_id.

Esta diferencia es importante: @ManyToOne y @OneToMany no crean dos columnas. Son dos vistas de la misma relación, implementada con una sola clave foránea en el lado del “muchos”.

T1.3 - Lado propietario y lado inverso

En JPA, toda relación tiene dos lados:

- Lado propietario (owning side): es el lado que controla la relación. Es el que tiene la clave foránea en la base de datos. Es el que se usa para persistir los cambios en la relación.
- Lado inverso (inverse side): es el lado que refleja la relación, pero no la controla. Se marca con el atributo mappedBy para indicar qué campo del lado propietario define la relación. En una relación @OneToMany / @ManyToOne:
- El lado @ManyToOne es el propietario. Tiene la clave foránea.
- El lado @OneToMany es el inverso. Se marca con mappedBy apuntando al campo del @ManyToOne.

Ejemplo:

```
@Entity
public class Alumno {

    @ManyToOne
    @JoinColumn(name = "curso_id")    // Lado propietario: tiene la FK
    private Curso curso;
}

@Entity
public class Curso {

    @OneToMany(mappedBy = "curso")    // Lado inverso: no tiene FK
    private List<Alumno> alumnos;
}
```

Regla: el lado @ManyToOne siempre es el propietario. El lado @OneToMany siempre es el inverso (salvo en relaciones unidireccionales, poco comunes). Consecuencia práctica: cuando se modifica

la relación (por ejemplo, se cambia un alumno de curso), hay que modificar el lado propietario (el @ManyToOne). Modificar solo el lado inverso no se refleja en la base de datos.

Pregunta de reflexión: ¿Por qué el lado @ManyToOne es el propietario? ¿Qué columna de la base de datos lo determina?

Bloque 2 - @ManyToOne

T2.1 - Estructura de @ManyToOne

Una relación @ManyToOne se declara en la entidad del “muchos” con la anotación @ManyToOne. Se combina con @JoinColumn para indicar el nombre de la columna de la clave foránea.

```
@Entity
@Table(name = "alumnos")
public class Alumno {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false, length = 100)
    private String nombre;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "curso_id", nullable = false)
    private Curso curso;

    // getters y setters
}
```

Detalles:

- @ManyToOne indica la cardinalidad.
- fetch = FetchType.LAZY indica que la relación se carga de forma perezosa. Es la opción recomendada: solo se carga el Curso cuando se accede a él.
- @JoinColumn(name = “curso_id”) indica el nombre de la columna de la clave foránea. Si no se pone, Hibernate genera un nombre por defecto (por ejemplo, curso_id).
- nullable = false indica que la relación es obligatoria: todo alumno debe tener un curso.

T2.2 - FetchType: EAGER vs LAZY

FetchType controla cuándo se carga la relación:

- EAGER: se carga inmediatamente al cargar la entidad. Se lanza un JOIN o una consulta adicional inmediatamente.
- LAZY: se carga solo cuando se accede al campo. Se lanza una consulta adicional en ese momento.

Por defecto:

- @ManyToOne y @OneToOne: EAGER.
- @OneToMany y @ManyToMany: LAZY.

Recomendación: usar siempre LAZY. El EAGER por defecto de @ManyToOne es una mala decisión de JPA que ha causado muchos problemas de rendimiento. Cargar siempre las relaciones de forma perezosa y decidir caso por caso qué cargar con JOIN FETCH.

Problema del EAGER: si cargas 100 alumnos con @ManyToOne(EAGER) a curso, Hibernate lanza 1 consulta para los alumnos y 100 para los cursos (una por alumno). Es el problema N+1.

Problema del LAZY fuera de transacción: si intentas acceder a una relación LAZY fuera de una transacción, Hibernate lanza LazyInitializationException porque la sesión ya está cerrada. Se soluciona cargando la relación dentro de la transacción con JOIN FETCH o transformando a DTO dentro de la transacción.

T2.3 - @JoinColumn en detalle

@JoinColumn personaliza la columna de la clave foránea. Sus atributos más importantes:

- name: el nombre de la columna. Por defecto, campo_id.
- nullable: si acepta nulos. Por defecto true. Si la relación es obligatoria, false.
- unique: si la columna debe ser única. Se usa en relaciones @OneToOne.
- foreignKey: personaliza la restricción de clave foránea.
- referencedColumnName: el nombre de la columna referenciada en la otra tabla. Por defecto, la clave primaria.

En la mayoría de los casos, solo se usa name y nullable. El resto se deja por defecto.

Pregunta de reflexión: ¿Por qué se recomienda FetchType.LAZY en @ManyToOne aunque el valor por defecto sea EAGER?

Bloque 3 - @OneToMany

T3.1 - Estructura de @OneToMany

Una relación @OneToMany se declara en la entidad del “uno” con la anotación @OneToMany. Se combina con mappedBy para indicar qué campo del lado propietario define la relación.

```
@Entity
@Table(name = "cursos")
public class Curso {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 50)
    private String nombre;

    @OneToMany(mappedBy = "curso", fetch = FetchType.LAZY)
    private List<Alumno> alumnos = new ArrayList<>();

    // getters y setters
}
```

Detalles:

- @OneToMany indica la cardinalidad.
- mappedBy = “curso” indica que la relación está mapeada por el campo curso de la entidad Alumno. Sin mappedBy, JPA crearía una tabla intermedia innecesaria.
- fetch = FetchType.LAZY es el valor por defecto, pero conviene ponerlo explícitamente.
- List alumnos = new ArrayList<>() se inicializa para evitar NullPointerException.

T3.2 - mappedBy y el lado inverso

mappedBy es crucial en @OneToMany. Indica que este lado no controla la relación: el control lo tiene el lado @ManyToOne de la otra entidad.

Si no se pone mappedBy, JPA asume que la relación es unidireccional con tabla intermedia. Eso significa que Hibernate crea una tabla adicional curso_alumnos con dos columnas (curso_id,

alumno_id) para gestionar la relación. Es un error común: crees que tienes una relación simple y acabas con una tabla intermedia innecesaria.

Ejemplo incorrecto:

```
@OneToMany // Incorrecto: sin mappedBy
private List<Alumno> alumnos;
```

Hibernate genera:

```
create table curso_alumnos (
    curso_id bigint not null,
    alumno_id bigint not null,
    primary key (curso_id, alumno_id)
)
```

Ejemplo correcto:

```
@OneToMany(mappedBy = "curso") // Correcto: con mappedBy
private List<Alumno> alumnos;
```

Hibernate usa la columna curso_id que ya existe en la tabla alumnos. No crea tabla intermedia.

Regla: siempre poner mappedBy en el lado @OneToMany cuando exista el lado @ManyToOne.

T3.3 - Métodos helper para mantener la relación

Cuando se modifica una relación bidireccional, hay que modificar ambos lados para que la entidad en memoria quede consistente. Si solo modificas el lado propietario, la base de datos se actualiza, pero la colección en memoria no refleja el cambio. Si solo modificas el lado inverso, la base de datos no se actualiza.

La solución es añadir métodos helper en el lado del “uno”:

```
@Entity
public class Curso {

    @OneToMany(mappedBy = "curso", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Alumno> alumnos = new ArrayList<>();

    public void addAlumno(Alumno alumno) {
        alumnos.add(alumno);
        alumno.setCurso(this);
    }

    public void removeAlumno(Alumno alumno) {
        alumnos.remove(alumno);
        alumno.setCurso(null);
    }
}
```

addAlumno añade el alumno a la lista (lado inverso) y establece el curso del alumno (lado propietario). Así ambos lados quedan consistentes.

removeAlumno hace lo contrario.

Estos métodos son la forma recomendada de gestionar relaciones bidireccionales. Evitan que se olvide actualizar uno de los lados. > **Pregunta de reflexión:** ¿Por qué hay que modificar ambos lados de una relación bidireccional? ¿Qué pasa si solo se modifica uno?

Bloque 4 - Cascade y orphanRemoval

T4.1 - CascadeType

cascade indica qué operaciones se propagan del lado propietario al lado inverso. Los valores posibles son:

- CascadeType.PERSIST: al persistir el padre, se persisten los hijos.
- CascadeType.MERGE: al hacer merge del padre, se hace merge de los hijos.
- CascadeType.REMOVE: al eliminar el padre, se eliminan los hijos.
- CascadeType.REFRESH: al hacer refresh del padre, se hace refresh de los hijos.
- CascadeType.DETACH: al hacer detach del padre, se hace detach de los hijos.
- CascadeType.ALL: todos los anteriores.

Ejemplo:

```
@OneToMany(mappedBy = "curso", cascade = CascadeType.ALL, orphanRemoval = true)
private List<Alumno> alumnos = new ArrayList<>();
```

Con cascade = CascadeType.ALL, al persistir un Curso se propagan las operaciones a sus alumnos; al eliminar el curso también puede propagarse la eliminación. Por eso debe usarse con criterio.

Cuidado: CascadeType.REMOVE es peligroso. Si eliminas un Curso, se eliminan todos sus alumnos. Si un alumno pudiera estar en varios cursos (relación muchos a muchos), sería catastrófico. Usar con criterio.

T4.2 - orphanRemoval

orphanRemoval es una opción que indica que, si un hijo se desvincula del padre (se quita de la colección), se elimina de la base de datos. Es útil para relaciones de composición, donde el hijo no tiene sentido sin el padre.

Ejemplo:

```
@OneToMany(mappedBy = "curso", cascade = CascadeType.ALL, orphanRemoval = true)
private List<Alumno> alumnos = new ArrayList<>();
```

Con orphanRemoval = true, si se elimina un alumno de la colección y se sincroniza la relación, Hibernate elimina el huérfano al hacer flush. Diferencia con CascadeType.REMOVE:

- CascadeType.REMOVE: al eliminar el padre, se eliminan los hijos.
- orphanRemoval = true: al desvincular un hijo del padre, se elimina el hijo.

orphanRemoval = true implica CascadeType.REMOVE en la práctica. La diferencia es que orphanRemoval también elimina al desvincular.

T4.3 - Cuándo usar cascade y orphanRemoval

La elección depende del tipo de relación:

Composición (el hijo no tiene sentido sin el padre):

- Ejemplo: un expediente y sus documentos. Un documento no existe sin el expediente.
- Usar: cascade = CascadeType.ALL, orphanRemoval = true.
- Al eliminar el expediente, se eliminan los documentos. Al quitar un documento del expediente, se elimina.

Agregación (el hijo puede existir sin el padre):

- Ejemplo: un alumno y su curso. Un alumno puede cambiar de curso, pero sigue existiendo.
- Usar: cascade = {} (sin cascade) y orphanRemoval = false.
- Al eliminar el curso, no se eliminan los alumnos. Al cambiar de curso, el alumno sigue existiendo.

Referencia (el hijo es independiente):

- Ejemplo: un alumno y su centro. El centro existe independientemente de los alumnos.
- Usar: cascade = {} y orphanRemoval = false.
- Al eliminar el centro, hay que decidir qué pasa con los alumnos (normalmente, restricción).

Pregunta de reflexión: ¿Qué diferencia hay entre CascadeType.REMOVE y orphanRemoval = true? ¿Cuándo usarías cada uno?

Bloque 5 - Rendimiento y problemas comunes

T5.1 - El problema N+1

El problema N+1 ocurre cuando se carga una lista de entidades (1 consulta) y, por cada una, se lanza una consulta adicional para cargar una relación (N consultas). Total: N+1 consultas.

Ejemplo: cargas 100 alumnos y accedes a alumno.getCurso().getNombre() para cada uno. Hibernate lanza: - 1 consulta para los 100 alumnos. - 100 consultas para los cursos (una por alumno).

Total: 101 consultas. Con 1000 alumnos, 1001 consultas. El rendimiento se degrada linealmente.

Cómo detectarlo: activar spring.jpa.show-sql=true y observar los logs. Si ves muchas consultas repetidas, tienes un N+1.

Cómo solucionarlo:

- JOIN FETCH en JPQL: carga la relación en la misma consulta.

```
@Query("SELECT a FROM Alumno a JOIN FETCH a.curso WHERE a.curso.id = :cursoId")
List<Alumno> findByCursoIdConCurso(@Param("cursoId") Long cursoId);
```

- @EntityGraph: anotación que indica qué relaciones cargar.

```
@EntityGraph(attributePaths = {"curso"})
List<Alumno> findByCursoId(Long cursoId);
```

- @BatchSize: carga las relaciones en lotes de N.

```
@BatchSize(size = 20)
@OneToMany(mappedBy = "curso")
private List<Alumno> alumnos;
```

En la práctica, JOIN FETCH y @EntityGraph son las soluciones más usadas.

T5.2 - LazyInitializationException

LazyInitializationException es el error más común al trabajar con relaciones LAZY. Ocurre cuando intentas acceder a una relación LAZY fuera de la transacción (o de la sesión de Hibernate).

Causa: cuando la transacción termina, Hibernate cierra la sesión. La entidad queda en estado detached. Si intentas acceder a una relación LAZY, Hibernate no puede lanzar la consulta porque no hay sesión.

Ejemplo típico:

```
@GetMapping("/{id}")
public AlumnoDTO consultar(@PathVariable Long id) {
    Alumno alumno = service.buscarEntidad(id); // La transacción termina aquí
    // toDTO accede a alumno.getCurso().getNombre().
    // Fuera de la sesión puede provocar LazyInitializationException.
    return toDTO(alumno);
}
```

Soluciones:

- Cargar la relación dentro de la transacción con JOIN FETCH o @EntityGraph.
- Transformar a DTO dentro de la transacción (en el servicio, no en el controlador).

- Usar @Transactional en el método que accede a la relación (pero eso extiende la transacción al controlador, no recomendado).

La mejor solución es transformar a DTO en el servicio, dentro de la transacción. Así el controlador nunca ve entidades y no hay riesgo de LazyInitializationException.

T5.3 - Buenas prácticas con relaciones

Usar FetchType.LAZY siempre. Aunque el valor por defecto de @ManyToOne sea EAGER, conviene poner LAZY explícitamente.

Poner mappedBy en el lado @OneToMany. Sin él, Hibernate crea una tabla intermedia innecesaria.

Usar métodos helper para mantener la relación. addAlumno y removeAlumno evitan inconsistencias.

Decidir cascade con cuidado. CascadeType.ALL es cómodo pero peligroso. Usar solo cuando la composición lo justifique.

Usar orphanRemoval en composición. Cuando el hijo no tiene sentido sin el padre.

Evitar el N+1 con JOIN FETCH o @EntityGraph. Cuando cargues relaciones en bucles.

Transformar a DTO dentro de la transacción. Evita LazyInitializationException. No exponer entidades en la API. Siempre DTOs.

Modelar relaciones unidireccionales cuando sea posible. Si no necesitas navegar en ambas direcciones, usa solo @ManyToOne. Es más simple y eficiente.

Pregunta de reflexión: ¿Por qué se recomienda modelar relaciones unidireccionales cuando sea posible? ¿Qué ventaja tiene frente a las bidireccionales?

Resumen de la teoría

- Cuatro tipos de relaciones: @OneToOne, @OneToMany, @ManyToOne, @ManyToMany.
- @ManyToOne: lado propietario. Tiene la clave foránea. FetchType.LAZY.
- @OneToMany: lado inverso. Usa mappedBy. FetchType.LAZY.
- mappedBy: obligatorio en el lado @OneToMany para evitar tabla intermedia.
- Métodos helper: addAlumno, removeAlumno para mantener ambos lados consistentes.
- cascade: PERSIST, MERGE, REMOVE, REFRESH, DETACH, ALL.
- orphanRemoval: elimina al desvincular.
- Problema N+1: 1 + N consultas. Se soluciona con JOIN FETCH o @EntityGraph.
- LazyInitializationException: acceso a relación LAZY fuera de transacción. Se soluciona transformando a DTO en el servicio.
- Buenas prácticas: LAZY, mappedBy, métodos helper, cascade con cuidado, DTOs.

Criterio profesional añadido

En Spring Boot 3.x la persistencia usa el espacio de nombres jakarta.persistence. La transacción pertenece a la capa de servicio, las entidades no deben ser el contrato HTTP y cualquier consulta potencialmente costosa debe comprobarse con SQL/logs y pruebas de repositorio.

Punto 4.5 - Consultas básicas

Objetivos de aprendizaje

1. Escribir consultas JPQL en profundidad: SELECT, WHERE, ORDER BY.
2. Usar parámetros posicionales y nombrados correctamente.
3. Aplicar joins en JPQL: INNER JOIN, LEFT JOIN y JOIN FETCH.
4. Usar funciones de string, numéricas y de fecha en JPQL.

5. Escribir agregaciones con GROUP BY y HAVING.
6. Devolver DTOs directamente desde consultas con proyecciones.
7. Diagnosticar y resolver los errores más comunes al escribir consultas.

Bloque 1 - JPQL en profundidad

T1.1 - Qué es JPQL y su relación con SQL

JPQL son las siglas de Java Persistence Query Language. Es un lenguaje de consulta orientado a objetos, definido por la especificación JPA. Su sintaxis se parece a SQL, pero hay una diferencia fundamental: JPQL trabaja con entidades y campos, no con tablas y columnas.

Cuando escribes SELECT a FROM Alumno a WHERE a.curso = :curso, estás diciendo:

- Alumno es el nombre de la entidad, no de la tabla.
- a es un alias para la entidad.
- a.curso es el nombre de un campo de la entidad, no de una columna.
- :curso es un parámetro nombrado.

Hibernate traduce esa consulta JPQL a SQL. Para la base de datos, genera algo como:

```
SELECT a.* FROM alumnos a WHERE a.curso_id = ?
```

La ventaja de JPQL es que es portable: si cambias de base de datos, la consulta JPQL sigue funcionando. El SQL generado será distinto, pero tú no tienes que cambiarlo.

Otra diferencia importante: en JPQL, los nombres son sensibles a mayúsculas para las entidades y los campos. Alumno no es lo mismo que alumno. En SQL, no importa. Esto es porque JPQL trabaja con clases Java, y Java es sensible a mayúsculas.

T1.2 - Estructura de una consulta JPQL

Una consulta JPQL tiene la misma estructura que una consulta SQL:

```
SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ...
```

Cada cláusula tiene su función:

SELECT: indica qué se devuelve. Puede ser:

- Una entidad completa: SELECT a FROM Alumno a.
- Varios campos: SELECT a.nombre, a.apellidos FROM Alumno a.
- Una agregación: SELECT COUNT(a) FROM Alumno a.
- Un constructor de DTO: SELECT new es.mecd.dto.AlumnoResumen(a.id, a.nombre) FROM Alumno a.

FROM: indica de qué entidad se consulta. Puede incluir un alias: FROM Alumno a. El alias es opcional si solo se usa la entidad, pero se recomienda ponerlo siempre.

WHERE: filtra los resultados. Se pueden combinar condiciones con AND, OR, NOT.

GROUP BY: agrupa los resultados por uno o varios campos. Se usa con agregaciones.

HAVING: filtra los grupos después de la agrupación. Se diferencia de WHERE en que WHERE filtra filas antes de agrupar, y HAVING filtra grupos después.

ORDER BY: ordena los resultados. Se puede ordenar por uno o varios campos, en orden ascendente (ASC) o descendente (DESC).

Un ejemplo completo:

```
@Query("""
SELECT a FROM Alumno a
WHERE a.curso = :curso AND a.estado = :estado
ORDER BY a.apellidos ASC, a.nombre ASC
```

```

""")
    List<Alumno> buscarPorCursoYEstado(
        @Param("curso") String curso,
        @Param("estado") EstadoAlumno estado);

```

Este ejemplo usa tres cláusulas: SELECT, FROM, WHERE y ORDER BY. Devuelve las entidades Alumno que cumplen las condiciones, ordenadas por apellidos y luego por nombre.

T1.3 - Parámetros posicionales y nombrados

En JPQL, los parámetros de una consulta se pueden pasar de dos formas:

Parámetros posicionales: se identifican por su posición, empezando en 1.

```

@Query("SELECT a FROM Alumno a WHERE a.curso = ?1 AND a.estado = ?2")
List<Alumno> buscar(String curso, EstadoAlumno estado);

```

Parámetros nombrados: se identifican por un nombre precedido de :.

```

@Query("SELECT a FROM Alumno a WHERE a.curso = :curso AND a.estado = :estado")
List<Alumno> buscar(
    @Param("curso") String curso,
    @Param("estado") EstadoAlumno estado);

```

Los nombrados son preferibles por varias razones:

- Claridad: :curso es más legible que ?1. Al leer la consulta, sabes qué parámetro es.
- Flexibilidad: si cambias el orden de los parámetros en el método, la consulta sigue funcionando.
- Mantenibilidad: si añades un parámetro en medio, no tienes que reenumerar los demás.

La única razón para usar posicionales es cuando hay muchos parámetros y no quieres repetir @Param. Pero en la práctica, casi siempre se usan nombrados.

Un detalle importante: en JPQL, los parámetros no llevan comillas. :curso es un parámetro; '5º' es un literal. Si escribes a.curso = '5º', estás comparando con un valor fijo. Si escribes a.curso = :curso, estás usando un parámetro.

Pregunta de reflexión: ¿Por qué los parámetros nombrados son preferibles a los posicionales? ¿Qué ventaja tienen al refactorizar el método?

Bloque 2 - Joins en JPQL

T2.1 - Tipos de joins

En JPQL, hay varios tipos de joins: - INNER JOIN (o solo JOIN): devuelve solo los registros que tienen correspondencia en ambas entidades. Si un alumno no tiene curso, no aparece. - LEFT JOIN: devuelve todos los registros de la entidad de la izquierda, y los de la derecha si existen. Si un alumno no tiene curso, aparece con curso = null. - CROSS JOIN: producto cartesiano. Poco usado en la práctica.

Un ejemplo con INNER JOIN:

```

@Query("SELECT a FROM Alumno a JOIN a.curso c WHERE c.nombre = :nombre")
List<Alumno> buscarPorNombreCurso(@Param("nombre") String nombre);

```

Un ejemplo con LEFT JOIN:

```

@Query("SELECT c FROM Curso c LEFT JOIN c.alumnos a WHERE a IS NULL")
List<Curso> buscarCursosSinAlumnos();

```

Este último devuelve los cursos que no tienen ningún alumno. Es un caso típico de LEFT JOIN: quieres todos los cursos, pero solo aquellos que no tienen alumnos.

La diferencia entre JOIN y LEFT JOIN es importante. JOIN es más restrictivo; LEFT JOIN es más permisivo. Usa JOIN cuando quieras solo las entidades que tienen relación; usa LEFT JOIN cuando quieras todas las entidades, tengan o no relación.

T2.2 - JOIN vs JOIN FETCH

En JPQL hay dos formas de hacer un join:

JOIN: se usa en la cláusula WHERE para filtrar. La entidad relacionada no se carga en la consulta principal. Si accedes a ella después, Hibernate lanza una consulta adicional.

```
@Query("SELECT a FROM Alumno a JOIN a.curso c WHERE c.nombre = :nombre")
List<Alumno> buscarPorCurso(@Param("nombre") String nombre);
```

Con este JOIN, Hibernate genera:

```
SELECT a.* FROM alumnos a INNER JOIN cursos c ON a.curso_id = c.id WHERE c.nombre = ?
```

Cuando accedas a alumno.getCurso(), Hibernate lanzará otra consulta para cargar el curso. Es el problema N+1.

JOIN FETCH: se usa en la cláusula SELECT para cargar la entidad relacionada en la misma consulta. La relación se carga de forma ansiosa en esa consulta concreta.

```
@Query("SELECT a FROM Alumno a JOIN FETCH a.curso WHERE a.curso.nombre = :nombre")
List<Alumno> buscarPorCursoConCurso(@Param("nombre") String nombre);
```

Con JOIN FETCH, Hibernate genera:

```
SELECT a.*, c.*
FROM alumnos a
INNER JOIN cursos c ON a.curso_id = c.id
WHERE c.nombre = ?
```

Ahora, cuando accedas a alumno.getCurso(), ya está cargado. No hay consulta adicional.

Regla: usa JOIN FETCH cuando vayas a acceder a la relación después. Usa JOIN cuando solo quieras filtrar por la relación pero no necesites cargarla.

T2.3 - Cuándo usar cada tipo de join

La elección depende del caso de uso:

JOIN simple: cuando solo necesitas filtrar por un campo de la entidad relacionada, pero no vas a acceder a esa entidad.

```
@Query("SELECT a FROM Alumno a JOIN a.curso c WHERE c.nombre = :nombre")
List<Alumno> buscarPorNombreCurso(@Param("nombre") String nombre);
```

Aquí solo devuelves alumnos; no accedes al curso.

JOIN FETCH: cuando necesitas la entidad relacionada para mostrarla o para transformarla a DTO.

```
@Query("SELECT a FROM Alumno a JOIN FETCH a.curso WHERE a.curso.nombre = :nombre")
List<Alumno> buscarPorNombreCursoConCurso(@Param("nombre") String nombre);
```

Aquí devuelves alumnos con su curso cargado, listo para transformar a DTO.

LEFT JOIN FETCH: cuando quieres todas las entidades, tengan o no relación, y cargar la relación si existe.

```
@Query("SELECT c FROM Curso c LEFT JOIN FETCH c.alumnos")
List<Curso> buscarTodosConAlumnos();
```

Aquí devuelves todos los cursos, con sus alumnos cargados (o lista vacía si no tienen).

Cuidado con JOIN FETCH en colecciones. Si haces JOIN FETCH sobre una relación @OneToMany, Hibernate puede devolver filas duplicadas (una por cada elemento de la colección). Se soluciona con DISTINCT:

```
@Query("SELECT DISTINCT c FROM Curso c LEFT JOIN FETCH c.alumnos")
List<Curso> buscarTodosConAlumnos();
```

Sin DISTINCT, obtendrías un curso repetido por cada alumno. > **Pregunta de reflexión:** ¿Por qué JOIN FETCH sobre una colección devuelve filas duplicadas? ¿Cómo se soluciona?

Bloque 3 - Funciones y expresiones

T3.1 - Funciones de string

JPQL ofrece varias funciones para manipular strings, similares a las de SQL:

- CONCAT(str1, str2, ...): concatena strings.
- UPPER(str): convierte a mayúsculas.
- LOWER(str): convierte a minúsculas.
- SUBSTRING(str, inicio, longitud): extrae una subcadena.
- TRIM(str): elimina espacios al principio y al final.
- LENGTH(str): devuelve la longitud.
- LOCATE(substr, str): devuelve la posición de una subcadena.

Ejemplos:

```
@Query("SELECT a FROM Alumno a WHERE LOWER(a.nombre) = LOWER(:nombre)")
List<Alumno> buscarPorNombreIgnoreCase(@Param("nombre") String nombre);
```

```
@Query(
    "SELECT a FROM Alumno a "
    + "WHERE CONCAT(a.nombre, ' ', a.apellidos) LIKE %:texto%"
)
List<Alumno> buscarPorNombreCompleto(@Param("texto") String texto);
```

```
@Query("SELECT a FROM Alumno a WHERE LENGTH(a.dni) = 9")
List<Alumno> buscarDniValido();
```

LOWER y UPPER son útiles para búsquedas que ignoran mayúsculas. CONCAT permite combinar campos para buscar por nombre completo. LENGTH permite validar longitudes.

T3.2 - Funciones numéricas y de fecha

JPQL también ofrece funciones para números y fechas:

Numéricas:

- ABS(num): valor absoluto.
- SQRT(num): raíz cuadrada.
- MOD(num, divisor): resto de la división.
- SIZE(coleccion): número de elementos de una colección. De fecha:
- CURRENT_DATE: fecha actual.
- CURRENT_TIME: hora actual.
- CURRENT_TIMESTAMP: fecha y hora actual.
- YEAR(fecha): año.

- MONTH(fecha): mes.
- DAY(fecha): día del mes.
- HOUR(fecha): hora.
- MINUTE(fecha): minuto.

Ejemplos:

```
@Query("SELECT a FROM Alumno a WHERE YEAR(a.fechaNacimiento) = :anio")
List<Alumno> buscarPorAnioNacimiento(@Param("anio") int anio);
```

```
@Query("SELECT a FROM Alumno a WHERE a.fechaNacimiento > :fecha")
List<Alumno> buscarNacidosDespues(@Param("fecha") LocalDate fecha);
```

```
@Query("SELECT c FROM Curso c WHERE SIZE(c.alumnos) > :minimo")
List<Curso> buscarCursosConMasDe(@Param("minimo") int minimo);
```

YEAR, MONTH y DAY son útiles para filtros por partes de una fecha. SIZE es útil para filtrar por el número de elementos de una colección.

T3.3 - Expresiones condicionales

JPQL permite usar expresiones condicionales con CASE WHEN:

```
@Query("""
    SELECT a FROM Alumno a
    WHERE (
        CASE WHEN :curso IS NULL THEN true
        ELSE a.curso.nombre = :curso
        END
    ) = true
""")
List<Alumno> buscarPorCursoOpcional(@Param("curso") String curso);
```

En este ejemplo, si :curso es null, la condición es true y no se filtra. Si no, se filtra por el curso. Los CASE WHEN son útiles para consultas con filtros opcionales. Pero en la práctica, cuando hay muchos filtros opcionales, las Specifications son más cómodas. Los CASE WHEN se usan para casos puntuales.

Pregunta de reflexión: ¿Cuándo usarías CASE WHEN en lugar de Specifications? ¿Qué ventaja tiene cada enfoque?

Bloque 4 - Agregaciones y proyecciones

T4.1 - Funciones de agregación

JPQL ofrece las mismas funciones de agregación que SQL:

- COUNT(expr): cuenta el número de elementos.
- COUNT(DISTINCT expr): cuenta los valores distintos.
- SUM(expr): suma los valores.
- AVG(expr): media.
- MIN(expr): mínimo.
- MAX(expr): máximo.

Ejemplos:

```
@Query("SELECT COUNT(a) FROM Alumno a WHERE a.curso = :curso")
long contarPorCurso(@Param("curso") String curso);
```

```
@Query("SELECT AVG(a.importe) FROM Expediente a WHERE a.estado = 'RESUELTA'")
```

```
Double mediaImporteResueltos();
```

```
@Query("SELECT MAX(a.importe) FROM Expediente a")
Double importeMaximo();
```

El resultado de una agregación es un valor único, no una lista. Por eso el tipo de retorno es long, Double, BigDecimal, etc.

Si la consulta no encuentra filas, COUNT devuelve 0, pero AVG, MIN y MAX devuelven null. Hay que tenerlo en cuenta.

T4.2 - GROUP BY y HAVING

Cuando se agrupa por un campo, se usa GROUP BY. Y cuando se filtra por el resultado de una agregación, se usa HAVING:

```
@Query("""
    SELECT a.curso, COUNT(a)
    FROM Alumno a
    GROUP BY a.curso
    HAVING COUNT(a) > :minimo
""")
List<Object[]> contarPorCursoConMinimo(@Param("minimo") long minimo);
```

El resultado es una lista de arrays Object[], donde cada array tiene dos elementos: el curso y el count.

WHERE vs HAVING:

- WHERE: filtra las filas antes de agrupar.
- HAVING: filtra los grupos después de agrupar.

Un ejemplo con ambos:

```
@Query("""
    SELECT a.curso, COUNT(a)
    FROM Alumno a
    WHERE a.estado = 'ACTIVO'
    GROUP BY a.curso
    HAVING COUNT(a) > :minimo
""")
List<Object[]> contarActivosPorCurso(@Param("minimo") long minimo);
```

Aquí primero se filtran los alumnos activos (WHERE), luego se agrupan por curso (GROUP BY), y finalmente se filtran los cursos que tienen más de :minimo alumnos (HAVING).

T4.3 - Proyecciones: devolver DTOs desde consultas

Devolver Object[] desde una consulta es incómodo. En su lugar, se pueden devolver DTOs directamente con la sintaxis new:

```
@Query("""
    SELECT new es.mecd.demo.miproyecto.alumno.AlumnoResumenDTO(
        a.id, a.nombre, a.apellidos, a.curso.nombre)
    FROM Alumno a
    WHERE a.estado = :estado
""")
List<AlumnoResumenDTO> buscarResumenPorEstado(
    @Param("estado") EstadoAlumno estado);
```

La expresión new + ruta completa del DTO + argumentos del constructor hace que Hibernate llame al constructor con esos argumentos y cree el DTO.

El DTO debe tener un constructor con los parámetros en el orden indicado:

```
public class AlumnoResumenDTO {
    private Long id;
    private String nombre;
    private String apellidos;
    private String curso;

    public AlumnoResumenDTO(Long id, String nombre, String apellidos, String curso) {
        this.id = id;
        this.nombre = nombre;
        this.apellidos = apellidos;
        this.curso = curso;
    }

    // getters
}
```

Las proyecciones son útiles cuando:

- Solo necesitas algunos campos, no la entidad completa.
- Quieres evitar cargar relaciones que no vas a usar.
- Quieres devolver datos calculados o agregados.

Ventaja: eficiencia. Solo se cargan las columnas necesarias, y no hay problema N+1.

Inconveniente: el DTO debe tener un constructor con la firma exacta. Si cambias el orden de los argumentos, la consulta falla. > **Pregunta de reflexión:** ¿Qué ventaja tiene devolver un DTO directamente desde una consulta en lugar de devolver la entidad y transformarla en el servicio?

Bloque 5 - Buenas prácticas y errores

T5.1 - Buenas prácticas con consultas JPQL

Usar parámetros nombrados. Son más legibles y mantenibles que los posicionales.

Usar JOIN FETCH cuando vayas a acceder a la relación. Evita el N+1.

Usar DISTINCT con JOIN FETCH sobre colecciones. Evita filas duplicadas.

Usar proyecciones cuando solo necesites algunos campos. Evita cargar datos innecesarios.

No concatenar strings para construir consultas. Usa parámetros. Concatenar abre la puerta a inyección de JPQL.

Documentar las consultas complejas. Un comentario explicando qué hace y por qué.

Limitar el tamaño de las consultas. Si una consulta devuelve 100.000 filas, algo va mal. Usar @Query con nombre en la entidad. Se puede definir la consulta en la entidad con @NamedQuery y referenciarla desde el repositorio. Es útil cuando la misma consulta se usa en varios sitios.

T5.2 - Errores comunes

Error 1: olvidar el alias en FROM.

SELECT a FROM Alumno WHERE a.curso = :curso falla porque a no está definido. Hay que poner FROM Alumno a.

Error 2: usar el nombre de la tabla en lugar de la entidad.

SELECT a FROM alumnos a falla porque alumnos es el nombre de la tabla, no de la entidad. Hay que usar Alumno.

Error 3: usar el nombre de la columna en lugar del campo.

SELECT a FROM Alumno a WHERE a.curso_id = :id falla porque curso_id es el nombre de la columna, no del campo. Hay que usar a.curso.id.

Error 4: olvidar @Param en los parámetros nombrados. Si la consulta usa :curso y el método tiene un parámetro String curso, Spring puede inferir el nombre. Pero si el compilador no conserva los nombres, falla. Mejor poner @Param("curso") explícitamente.

Error 5: devolver Object[] en lugar de un DTO.

Es incómodo y propenso a errores. Mejor usar proyecciones con new.

Error 6: olvidar @Modifying en consultas de actualización.

Sin @Modifying, Spring Data espera una consulta de selección y falla al ejecutar.

Error 7: ejecutar @Modifying sin transacción.

Lanza TransactionRequiredException. Hay que llamar al método desde un servicio con @Transactional.

Error 8: usar JOIN FETCH sobre dos colecciones a la vez.

Hibernate lanza MultipleBagFetchException porque no puede hacer JOIN FETCH de dos bolsas (listas sin orden) a la vez. Se soluciona usando Set en lugar de List, o haciendo dos consultas separadas.

Error 9: olvidar DISTINCT con JOIN FETCH sobre colecciones.

Devuelve filas duplicadas. Error 10: consultas con LIKE sin comodines.

WHERE a.nombre LIKE :texto con texto = "Ana" solo encuentra "Ana" exacto. Para buscar coincidencias parciales, hay que añadir comodines: LIKE %:texto% o LIKE CONCAT('%', :texto, '%').

T5.3 - Rendimiento y optimización

Usar EXPLAIN para analizar consultas. En PostgreSQL, EXPLAIN ANALYZE muestra el plan de ejecución. En MySQL, EXPLAIN. Es útil para detectar consultas lentas.

Indexar las columnas de filtro. Si filtras por curso, la columna curso_id debería tener un índice. Se define en la base de datos, no en JPA (aunque JPA puede crearlo con @Table(indexes = ...)).

Evitar SELECT * en consultas nativas. Cargar columnas que no se usan es ineficiente.

Usar paginación siempre en colecciones grandes. No devolver 100.000 filas de golpe.

Cachear consultas frecuentes. JPA ofrece caché de segundo nivel con @Cacheable. Se configura en la entidad y en el application.properties. Medir antes de optimizar. No optimices por intuición. Mide con spring.jpa.show-sql=true y observa los tiempos.

Pregunta de reflexión: ¿Por qué es importante medir antes de optimizar? ¿Qué herramienta usarías para medir el rendimiento de una consulta?

Resumen de la teoría

- JPQL: lenguaje de consulta orientado a objetos. Trabaja con entidades y campos.
- Estructura: SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY.
- Parámetros: nombrados (:curso) preferibles a posicionales (?1).
- Joins: INNER JOIN, LEFT JOIN, CROSS JOIN.
- JOIN vs JOIN FETCH: el segundo carga la relación en la misma consulta.
- DISTINCT con JOIN FETCH: evita duplicados en colecciones.
- Funciones: CONCAT, UPPER, LOWER, SUBSTRING, YEAR, MONTH, SIZE.
- CASE WHEN: expresiones condicionales.
- Agregaciones: COUNT, SUM, AVG, MIN, MAX.
- GROUP BY y HAVING:
- Proyecciones: devolver DTOs con new.

- Buenas prácticas: parámetros nombrados, JOIN FETCH, proyecciones, medir.

Criterio profesional añadido

En Spring Boot 3.x la persistencia usa el espacio de nombres `jakarta.persistence`. La transacción pertenece a la capa de servicio, las entidades no deben ser el contrato HTTP y cualquier consulta potencialmente costosa debe comprobarse con SQL/logs y pruebas de repositorio.

Punto 4.6 - Gestión de transacciones

Objetivos de aprendizaje

1. Explicar qué es una transacción y las propiedades ACID.
2. Usar `@Transactional` correctamente en el servicio.
3. Entender los tipos de propagación y cuándo usar cada uno.
4. Conocer los niveles de aislamiento y los problemas de concurrencia.
5. Configurar el rollback ante excepciones `checked` y `unchecked`.
6. Diagnosticar y resolver los errores más comunes con transacciones.

Bloque 1 - Qué es una transacción

T1.1 - Definición y propiedades ACID

Una transacción es un conjunto de operaciones sobre la base de datos que se ejecutan como una unidad atómica: o se ejecutan todas, o no se ejecuta ninguna. Si una falla, se deshacen todos los cambios realizados hasta ese momento. Si todas tienen éxito, se confirman.

Imagina que estás creando un expediente. Para crearlo, necesitas:

1. Insertar la fila del expediente en la tabla expedientes.
2. Insertar las filas de los documentos en la tabla documentos.
3. Actualizar el contador de expedientes del año.

Si algo falla después del paso 1 pero antes del paso 3, el sistema quedaría inconsistente: habría un expediente sin documentos y sin contador actualizado. Una transacción garantiza que eso no ocurra: o se ejecutan los tres pasos, o no se ejecuta ninguno.

Las transacciones tienen cuatro propiedades, conocidas como ACID:

Atomicidad (Atomicity): la transacción es una unidad indivisible. O se ejecuta todo, o no se ejecuta nada. Si falla un paso, se deshacen los anteriores. **Consistencia (Consistency):** la transacción lleva la base de datos de un estado válido a otro estado válido. No puede dejar la base de datos en un estado inconsistente. Por ejemplo, no puede dejar una clave foránea apuntando a una fila que no existe.

Aislamiento (Isolation): las transacciones concurrentes no se ven entre sí. Cada una ve la base de datos como si fuera la única. El nivel de aislamiento controla cuánto se aíslan.

Durabilidad (Durability): una vez confirmada una transacción, sus cambios son permanentes. Aunque el sistema se caiga justo después, los cambios siguen ahí.

Estas cuatro propiedades son la base de la fiabilidad de las bases de datos relacionales. Sin ellas, un sistema que gestiona expedientes, becas o matrículas sería un caos.

T1.2 - Transacciones en Spring

En Spring, las transacciones se gestionan de forma declarativa con la anotación `@Transactional`. No tienes que abrir y cerrar transacciones manualmente; Spring lo hace por ti.

Cuando un método anotado con `@Transactional` se invoca, Spring:

1. Abre una transacción antes de ejecutar el método.
2. Ejecuta el método.
3. Hace commit si el método termina normalmente.
4. Hace rollback si el método lanza una excepción (por defecto, si es RuntimeException).

Spring se apoya en un gestor de transacciones (normalmente JpaTransactionManager cuando se usa JPA) para interactuar con la base de datos. El gestor abre la conexión, inicia la transacción, la confirma o la revierte.

La ventaja de la gestión declarativa es que separa la lógica de negocio de la gestión transaccional. El servicio se centra en lo que hace; Spring se encarga de cómo lo hace.

T1.3 - @Transactional en detalle

La anotación @Transactional tiene varios atributos que controlan su comportamiento:

- propagation: cómo se comporta la transacción si ya hay una activa. Por defecto, REQUIRED.
- isolation: el nivel de aislamiento. Por defecto, el de la base de datos (normalmente READ_COMMITTED).
- readOnly: si la transacción es de solo lectura. Por defecto, false.
- timeout: el tiempo máximo en segundos antes de que la transacción se aborte. Por defecto, el del gestor.
- rollbackFor: excepciones que provocan rollback. Por defecto, RuntimeException y Error.
- noRollbackFor: excepciones que no provocan rollback.
- rollbackForClassName y noRollbackForClassName: versiones con nombre de clase.

Un ejemplo completo:

```
@Transactional(
    propagation = Propagation.REQUIRED,
    isolation = Isolation.READ_COMMITTED,
    readOnly = false,
    timeout = 30,
    rollbackFor = NegocioException.class
)
public AlumnoDTO crear(AlumnoRequestDTO request) {
    // ...
}
```

En la práctica, la mayoría de las veces solo se usa readOnly = true para consultas:

```
@Transactional(readOnly = true)
public List<AlumnoDTO> listar() {
    // ...
}
```

readOnly = true expresa que el método es de lectura y permite optimizaciones del contexto de persistencia. No debe entenderse como un mecanismo de autorización ni como una garantía absoluta de que nunca se emitirá una escritura.

Pregunta de reflexión: ¿Por qué readOnly = true mejora el rendimiento en consultas?
¿Qué deja de hacer Hibernate?

Bloque 2 - Propagación

T2.1 - Qué es la propagación

La propagación define cómo se comporta un método anotado con @Transactional cuando es invocado desde otro método que ya está dentro de una transacción. Es decir: ¿qué pasa si el método A (transaccional) llama al método B (también transaccional)?

Hay varias respuestas posibles:

- B se une a la transacción de A.
- B crea una nueva transacción independiente.
- B se ejecuta sin transacción.
- B falla si no hay transacción.

Cada respuesta corresponde a un tipo de propagación.

T2.2 - Tipos de propagación

Spring ofrece siete tipos de propagación:

REQUIRED (por defecto): si hay una transacción activa, el método se une a ella. Si no, crea una nueva. Es el más común: la mayoría de los métodos transaccionales comparten la transacción del llamador.

REQUIRES_NEW: siempre crea una nueva transacción, independientemente de si hay una activa. La transacción existente se suspende hasta que la nueva termine. Útil cuando quieres que una operación se ejecute aunque la transacción principal falle.

SUPPORTS: si hay una transacción activa, el método se une a ella. Si no, se ejecuta sin transacción. Útil para métodos que pueden o no necesitar transacción.

NOT_SUPPORTED: el método se ejecuta sin transacción, suspendiendo la existente si la hay. Útil para operaciones que no deben formar parte de una transacción (por ejemplo, llamadas a servicios externos).

MANDATORY: el método debe ejecutarse dentro de una transacción. Si no hay, lanza `IllegalTransactionStateException`. Útil para métodos que solo tienen sentido dentro de una transacción.

NEVER: el método no debe ejecutarse dentro de una transacción. Si hay, lanza `IllegalTransactionStateException`. Útil para métodos que no deben formar parte de una transacción. **NESTED**: ejecuta el método dentro de una transacción anidada (savepoint). Si el método falla, se revierte solo esa parte, no toda la transacción. Útil para operaciones que pueden fallar sin afectar al resto.

Los más usados son **REQUIRED** (por defecto) y **REQUIRES_NEW**. Los demás son para casos específicos.

T2.3 - Cuándo usar cada propagación

La elección depende del caso de uso:

REQUIRED: para la mayoría de los métodos. Si el llamador ya tiene una transacción, el método se une a ella. Así se garantiza que todas las operaciones se ejecuten como una unidad.

REQUIRES_NEW: cuando quieres que una operación se ejecute independientemente de la transacción principal. Por ejemplo, registrar una auditoría aunque la operación principal falle. Si la auditoría estuviera en la misma transacción, se revertiría con ella.

SUPPORTS: cuando el método puede ejecutarse con o sin transacción. Por ejemplo, un método de consulta que funciona igual dentro - fuera de una transacción.

NOT_SUPPORTED: cuando el método no debe formar parte de una transacción. Por ejemplo, una llamada a un servicio externo que no debe bloquear la transacción. **MANDATORY**: cuando el método solo tiene sentido dentro de una transacción. Por ejemplo, un método que modifica una entidad y necesita que el contexto de persistencia esté activo.

NEVER: cuando el método no debe ejecutarse dentro de una transacción. Por ejemplo, un método que hace una operación que no soporta transacciones.

NESTED: cuando quieres que una operación pueda fallar sin afectar al resto. Por ejemplo, insertar varios registros y que los fallidos se reviertan sin afectar a los exitosos.

Pregunta de reflexión: ¿Por qué **REQUIRED** es el tipo de propagación por defecto? ¿Qué problema evita?

Bloque 3 - Aislamiento y concurrencia

T3.1 - Niveles de aislamiento

El nivel de aislamiento controla cuánto se aíslan las transacciones concurrentes. Hay cuatro niveles, de menor a mayor aislamiento:

READ_UNCOMMITTED: una transacción puede leer cambios no confirmados de otra. Es el nivel más bajo y el más peligroso: permite lecturas sucias. **READ_COMMITTED:** una transacción solo lee cambios confirmados de otras. Es el nivel por defecto en PostgreSQL, Oracle y SQL Server. Evita las lecturas sucias, pero permite lecturas no repetibles y lecturas fantasma.

REPEATABLE_READ: una transacción ve los mismos datos a lo largo de su ejecución, aunque otras transacciones los modifiquen. Evita las lecturas no repetibles, pero permite lecturas fantasma. Es el nivel por defecto en MySQL.

SERIALIZABLE: el nivel más alto. Las transacciones se ejecutan como si fueran secuenciales. Evita todos los problemas de concurrencia, pero es el más lento. Se usa solo cuando la consistencia es crítica.

En Spring, se configura con isolation:

```
@Transactional(isolation = Isolation.SERIALIZABLE)
public void operacionCritica() {
    // ...
}
```

En la práctica, se usa el nivel por defecto de la base de datos. Cambiarlo solo cuando hay un problema específico.

T3.2 - Problemas de concurrencia

Cuando varias transacciones acceden a los mismos datos a la vez, pueden ocurrir varios problemas:

Lectura sucia (dirty read): una transacción lee un cambio no confirmado de otra. Si la otra hace rollback, la primera ha leído un valor que nunca existió. Ocurre en **READ_UNCOMMITTED**.

Lectura no repetible (non-repeatable read): una transacción lee una fila, otra transacción la modifica y confirma, y la primera vuelve a leer la misma fila y ve un valor distinto. Ocurre en **READ_COMMITTED** y **READ_UNCOMMITTED**.

Lectura fantasma (phantom read): una transacción ejecuta una consulta que devuelve un conjunto de filas, otra transacción inserta una fila que cumple la condición, y la primera vuelve a ejecutar la consulta y ve una fila nueva. Ocurre en **READ_COMMITTED**, **READ_UNCOMMITTED** y **REPEATABLE_READ**.

Pérdida de actualización (lost update): dos transacciones leen el mismo valor, ambas lo modifican y confirman. La segunda sobrescribe el cambio de la primera. Ocurre en todos los niveles salvo **SERIALIZABLE**.

Estos problemas son la razón por la que existen los niveles de aislamiento. Cuanto más alto el nivel, menos problemas, pero menos concurrencia.

T3.3 - Bloqueo optimista vs pesimista

Para evitar la pérdida de actualización, Spring ofrece dos estrategias de bloqueo:

Bloqueo optimista (optimistic locking): se añade un campo `@Version` a la entidad. Al actualizar, Hibernate incluye la versión en el `WHERE`. Si otra transacción ha modificado la entidad, la versión ha cambiado y el `UPDATE` no afecta a ninguna fila. Hibernate lanza `OptimisticLockException`. Es el bloqueo recomendado para APIs REST: no bloquea la base de datos y detecta conflictos.

Bloqueo pesimista (pessimistic locking): se bloquea la fila en la base de datos al leerla. Otras transacciones no pueden modificarla hasta que la primera termine. Se configura con `LockModeType.PESSIMISTIC_READ` o `PESSIMISTIC_WRITE` en una consulta. Es más seguro pero bloquea recursos, lo que reduce la concurrencia. Se usa cuando la probabilidad de conflicto es alta.

En Spring Data JPA, el bloqueo pesimista se aplica con `@Lock`:

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT a FROM Alumno a WHERE a.id = :id")
Optional<Alumno> findByIdConBloqueo(@Param("id") Long id);
```

Recomendación: usar bloqueo optimista para APIs REST (con `@Version`), y pesimista solo cuando la probabilidad de conflicto es muy alta y el coste de reintentar es elevado.

Pregunta de reflexión: ¿Por qué el bloqueo optimista es más adecuado para APIs REST que el pesimista?

Bloque 4 - Rollback y excepciones

T4.1 - Rollback por defecto

Cuando un método anotado con `@Transactional` lanza una excepción, Spring decide si hacer commit o rollback. La regla por defecto es:

- `RuntimeException` y `Error`: rollback.
- Excepciones checked (`Exception`): commit.

Esto sorprende a mucha gente. Si lanzas una `Exception checked` (por ejemplo, `IOException`), Spring no hace rollback. Confirma los cambios que se hayan hecho hasta ese momento.

¿Por qué? Porque las excepciones checked se consideran recuperables: el código que las captura puede decidir seguir adelante. Las `RuntimeException` se consideran no recuperables: algo ha ido mal y no hay forma de continuar.

En la práctica, la mayoría de las excepciones de negocio se modelan como `RuntimeException` (extendiendo `RuntimeException`, no `Exception`). Por eso el rollback se hace correctamente.

T4.2 - `rollbackFor` y `noRollbackFor`

Si quieres que una excepción checked provoque rollback, se usa `rollbackFor`:

```
@Transactional(rollbackFor = IOException.class)
public void procesarFichero() throws IOException {
    // ...
}
```

Si quieres que una `RuntimeException` no provoque rollback, se usa `noRollbackFor`:

```
@Transactional(noRollbackFor = NegocioAvisoException.class)
public void crearConAviso(AlumnoRequestDTO request) {
    // ...
}
```

`NegocioAvisoException` sería una excepción de negocio que no debe revertir la transacción. Por ejemplo, un aviso que no impide continuar.

La mayoría de las veces no hace falta especificar nada: con usar `RuntimeException` para las excepciones de negocio, el rollback se hace correctamente.

T4.3 - Excepciones checked vs unchecked

Esta distinción es importante y conviene tenerla clara:

Excepciones checked: extienden Exception (no RuntimeException). El compilador obliga a capturarlas o declararlas con throws. Ejemplos: IOException, SQLException, ParseException. Spring no hace rollback con ellas por defecto.

Excepciones unchecked: extienden RuntimeException. El compilador no obliga a capturarlas. Ejemplos: NullPointerException, IllegalArgumentException, NegocioException. Spring sí hace rollback con ellas por defecto.

Regla práctica: las excepciones de negocio (violación de reglas) se modelan como unchecked. Las excepciones técnicas (fallo de red, fichero no encontrado) pueden ser checked o unchecked, según el caso. Si quieres que una checked provoque rollback, usa rollbackFor.

Pregunta de reflexión: ¿Por qué las excepciones de negocio se modelan como RuntimeException en lugar de Exception?

Bloque 5 - Buenas prácticas y errores

T5.1 - Dónde poner @Transactional

@Transactional va en el servicio, no en el repositorio. El repositorio ya tiene transacciones por defecto en los métodos de JpaRepository, pero el servicio es el que define la unidad de trabajo.

No poner @Transactional en el controlador. El controlador no debe gestionar transacciones. Su responsabilidad es HTTP.

No poner @Transactional en métodos privados. Spring solo intercepta métodos públicos cuando usa proxies. Los métodos privados no se interceptan. Si necesitas transacción en un método privado, muévelo a otro bean o hazlo público.

No llamar a un método @Transactional desde otro método de la misma clase. Si lo haces, la llamada no pasa por el proxy de Spring y la transacción no se aplica. Se soluciona moviendo el método a otro bean o inyectando el propio bean.

Usar readOnly = true en consultas. Mejora el rendimiento.

Ser consistente. Si un servicio es transaccional, todos sus métodos que modifican datos deben serlo.

T5.2 - Errores comunes

Error 1: no poner @Transactional en un método que modifica datos.

Los cambios no se confirman, o se confirman parcialmente. Puede provocar inconsistencias. Error 2: llamar a un método @Transactional desde el mismo bean.

La transacción no se aplica porque la llamada no pasa por el proxy. Se soluciona moviendo el método a otro bean.

Error 3: usar @Transactional en un método privado.

Spring no intercepta métodos privados. La transacción no se aplica.

Error 4: esperar rollback con excepciones checked.

Por defecto, Spring no hace rollback con checked. Hay que usar rollbackFor.

Error 5: capturar la excepción dentro del método transaccional.

Si capturas la excepción y no la relanzas, Spring no sabe que ha habido un error y hace commit. Hay que relanzar la excepción (o marcar la transacción como rollback-only con TransactionAspectSupport.currentTransactionStatus().setRollbackOnly()).

Error 6: transacciones demasiado largas.

Una transacción que dura mucho tiempo bloquea recursos y reduce la concurrencia. Hay que hacer las transacciones lo más cortas posible.

Error 7: usar `REQUIRES_NEW` sin necesidad. Cada `REQUIRES_NEW` abre una nueva conexión y una nueva transacción. Es costoso. Usarlo solo cuando sea necesario.

Error 8: no tener en cuenta el bloqueo optimista.

Si dos transacciones modifican la misma entidad, la segunda puede sobrescribir a la primera sin saberlo. Usar `@Version`.

Error 9: mezclar `@Transactional` con operaciones no transaccionales.

Por ejemplo, llamar a un servicio externo dentro de una transacción. Si el servicio externo tarda, la transacción se alarga y bloquea recursos. Separar las operaciones.

Error 10: olvidar el `@Transactional` en métodos de test.

Los tests que modifican datos necesitan transacción. `@DataJpaTest` y `@SpringBootTest` la incluyen por defecto, pero los tests unitarios no.

T5.3 - Rendimiento

Las transacciones tienen un coste. Abrir y cerrar una transacción implica abrir y cerrar una conexión (o reutilizarla del pool), iniciar y confirmar la transacción en la base de datos, y liberar los recursos. Por eso conviene: Hacer las transacciones lo más cortas posible. No incluir operaciones lentas (llamadas a APIs externas, generación de PDFs) dentro de una transacción. Hacerlas antes o después.

Usar `readOnly = true` en consultas. Permite a Hibernate optimizar y a la base de datos usar conexiones de solo lectura.

No abrir transacciones innecesarias. Si una operación no modifica datos y no necesita consistencia, no hace falta transacción.

Usar `REQUIRES_NEW` solo cuando sea necesario. Cada `REQUIRES_NEW` es costoso.

Medir el tiempo de las transacciones. Con `spring.jpa.properties.hibernate.generate_statistics=true` se pueden ver métricas. Con Actuator y Micrometer, se pueden monitorizar.

Usar un pool de conexiones adecuado. HikariCP es el pool por defecto de Spring Boot. Configurar `maximum-pool-size` según la carga. Un pool demasiado pequeño limita la concurrencia; uno demasiado grande consume recursos.

Pregunta de reflexión: ¿Por qué es mala idea llamar a un servicio externo dentro de una transacción?

Resumen de la teoría

- Transacción: unidad atómica de operaciones. Propiedades ACID.
- `@Transactional`: gestión declarativa. Spring abre, ejecuta, confirma o revierte.
- Atributos: `propagation`, `isolation`, `readOnly`, `timeout`, `rollbackFor`.
- Propagación: `REQUIRED` (por defecto), `REQUIRES_NEW`, `SUPPORTS`, `NOT_SUPPORTED`, `MANDATORY`, `NEVER`, `NESTED`.
- Aislamiento: `READ_UNCOMMITTED`, `READ_COMMITTED`, `REPEATABLE_READ`, `SERIALIZABLE`.
- Problemas de concurrencia: lectura sucia, no repetible, fantasma, pérdida de actualización.
- Bloqueo: optimista (`@Version`) vs pesimista (`@Lock`).
- Rollback: `RuntimeException` y `Error` por defecto. `rollbackFor` para checked.
- Buenas prácticas: `@Transactional` en el servicio, métodos públicos, transacciones cortas, `readOnly` en consultas.
- Errores comunes: llamadas internas, métodos privados, capturar excepciones, transacciones largas.

Criterio profesional añadido

En Spring Boot 3.x la persistencia usa el espacio de nombres `jakarta.persistence`. La transacción pertenece a la capa de servicio, las entidades no deben ser el contrato HTTP y cualquier consulta potencialmente costosa debe comprobarse con SQL/logs y pruebas de repositorio.

Punto 4.7 - Testing de JPA con @DataJpaTest

Objetivos de aprendizaje

1. Explicar por qué el testing de JPA es distinto del testing unitario puro.
2. Configurar tests de repositorios con `@DataJpaTest`.
3. Usar `TestEntityManager` para preparar datos de prueba.
4. Probar consultas derivadas, `@Query` y `Specifications`.
5. Probar relaciones entre entidades y verificar el comportamiento de LAZY.
6. Verificar el rollback de transacciones en tests.
7. Diagnosticar y resolver los errores más comunes al testear JPA.

Bloque 1 - Por qué testear JPA es distinto

T1.1 - El problema de testear la persistencia

En el Módulo 2 aprendimos a testear servicios con Mockito. El patrón era: mockear el repositorio, ejecutar el método del servicio, verificar el resultado. Eso funciona muy bien para probar lógica de negocio, porque el repositorio es una dependencia que simulamos.

Pero cuando lo que queremos probar es el acceso a datos —las consultas del repositorio, el mapeo de entidades, las relaciones, las transacciones— mockear el repositorio no sirve. El repositorio es lo que queremos probar. Mockearlo sería como testear un coche con un motor simulado: no probaríamos el motor.

Para testear JPA necesitamos ejecutar las consultas contra una base de datos real. No la de producción, sino una base de datos de test, ligera y en memoria, que se levante al inicio del test y se destruya al final. Esa base de datos es H2, que ya conocemos del punto 4.1.

El testing de JPA, por tanto, es un test de integración: verifica que el repositorio, Hibernate, el dialecto de H2 y el mapeo de entidades funcionan juntos correctamente. Es más lento que un test unitario puro, pero sigue siendo rápido (segundos) porque H2 es en memoria.

T1.2 - La pirámide de tests aplicada a JPA

En el Módulo 2 vimos la pirámide de tests: muchos unitarios, algunos de integración, pocos end-to-end. Aplicada a JPA, la pirámide se ve así:

Tests unitarios de servicio: mockean el repositorio. Prueban la lógica de negocio. Son rápidos (milisegundos).

Tests de integración de repositorios (`@DataJpaTest`): usan H2 en memoria. Prueban las consultas, el mapeo y las relaciones. Son más lentos (segundos) pero siguen siendo rápidos.

Tests de integración completos (`@SpringBootTest`): arrancan toda la aplicación. Prueban el flujo completo: controlador, servicio, repositorio, base de datos. Son los más lentos (decenas de segundos) y se usan con moderación.

La mayoría de los tests de JPA son del segundo tipo: `@DataJpaTest`. Prueban el repositorio de forma aislada, sin arrancar el resto de la aplicación.

T1.3 - Qué se prueba en un test de JPA

Un test de JPA típico verifica:

El mapeo de entidades. Que los campos de la entidad se mapean correctamente a las columnas. Que los tipos coinciden. Que las restricciones (nullable, unique, length) se aplican.

Las consultas. Que los métodos derivados (findByCurso, existsByDni) generan las consultas correctas. Que las consultas con @Query devuelven lo esperado. Que las Specifications se combinan bien. Las relaciones. Que @ManyToOne y @OneToMany funcionan. Que el mappedBy está bien puesto. Que el FetchType.LAZY no se carga a menos que se acceda. Que el cascade y orphanRemoval se comportan como se espera.

Las transacciones. Que un método @Transactional hace rollback cuando falla. Que un @Modifying sin @Transactional falla. Que REQUIRES_NEW abre una transacción independiente.

Los constraints. Que un DNI duplicado lanza excepción. Que un campo nullable = false no acepta null. Que la longitud máxima se respeta.

En un test de JPA no se prueba la lógica de negocio (eso es del servicio) ni la capa HTTP (eso es del controlador). Solo el acceso a datos.

Pregunta de reflexión: ¿Por qué un test de JPA no puede usar Mockito para el repositorio? ¿Qué perdería?

Bloque 2 - @DataJpaTest en profundidad

T2.1 - Qué es @DataJpaTest

@DataJpaTest es una anotación de Spring Boot que configura un entorno de test para probar repositorios JPA de forma aislada. Cuando se aplica a una clase de test, Spring Boot:

1. Arranca solo la infraestructura JPA: entidades, repositorios, EntityManager, transacciones.
2. NO arranca el resto de la aplicación: no carga controladores, servicios, ni otros beans.
3. Configura una base de datos en memoria: por defecto, H2. No usa la base de datos real.
4. Configura una transacción por test: cada método de test se ejecuta dentro de una transacción que se revierte al final.
5. Proporciona TestEntityManager: una clase de ayuda para preparar datos de prueba.

La consecuencia principal es que los tests son rápidos y aislados. No hay que arrancar Tomcat, ni cargar todos los beans, ni preocuparse por el estado de la base de datos entre tests.

T2.2 - La transacción por test

La característica más importante de @DataJpaTest es que cada test se ejecuta dentro de una transacción que se revierte al final. Esto significa:

- Al empezar el test, la base de datos está vacía (o con los datos iniciales que hayas definido).
- El test puede insertar, modificar o eliminar datos sin miedo.
- Al terminar el test, todos los cambios se deshacen. La base de datos vuelve al estado inicial.

Esta transacción automática es la razón por la que los tests de JPA son tan cómodos: no tienes que limpiar la base de datos entre tests. Cada test empieza con un estado limpio.

Sin embargo, hay un matiz importante: la transacción del test envuelve el método de test. Si el test llama a un método con @Transactional(propagation = REQUIRES_NEW), esa transacción se abre dentro de la del test y se confirma independientemente. Es un caso poco común en tests, pero conviene saberlo. Si quieres desactivar la transacción automática (por ejemplo, para probar el comportamiento de una transacción real), se usa @Transactional(propagation = Propagation.NOT_SUPPORTED) en la clase de test. Entonces tienes que limpiar la base de datos manualmente entre tests.

T2.3 - TestEntityManager

TestEntityManager es una clase de ayuda que Spring Boot proporciona en los tests con @DataJpaTest. Envuelve al EntityManager de JPA y añade métodos convenientes para preparar datos de prueba.

Métodos principales:

- persist(Object entity): persiste una entidad y la devuelve. Es como save del repositorio, pero sin flush.
- persistAndFlush(Object entity): persiste y hace flush. Útil cuando necesitas que la entidad tenga ID antes de continuar.
- persistAndGetId(Object entity): persiste y devuelve el ID generado.
- flush(): hace flush de todos los cambios pendientes.
- clear(): limpia el contexto de persistencia. Útil para forzar una recarga desde la base de datos.
- find(Class, Object id): busca por ID sin pasar por el repositorio.
- merge(Object entity): hace merge de una entidad detached.

El TestEntityManager se inyecta con @Autowired en la clase de test. Se usa principalmente en el @BeforeEach para preparar datos:

```
@DataJpaTest
class AlumnoRepositoryTest {
    @Autowired
    private TestEntityManager entityManager;

    @Autowired
    private AlumnoRepository repositorio;

    private Curso curso;

    @BeforeEach
    void setUp() {
        curso = entityManager.persistAndFlush(new Curso("5º Primaria"));
    }
}
```

Pregunta de reflexión: ¿Por qué el TestEntityManager se usa en el @BeforeEach en lugar de usar el repositorio directamente?

Bloque 3 - Probar consultas

T3.1 - Probar métodos derivados

Los métodos derivados son los más fáciles de probar. Se prepara un conjunto de datos conocidos, se llama al método, y se verifica que devuelve lo esperado.

```
@Test
void findByCurso_debeDevolverAlumnosDelCurso() {
    Alumno ana = entityManager.persistAndFlush(
        new Alumno("Ana", "García", "12345678A", null, curso));
    entityManager.persistAndFlush(
        new Alumno("Luis", "Pérez", "87654321B", null, curso));
    entityManager.persistAndFlush(
        new Alumno("María", "López", "11111111C", null, otroCurso));

    List<Alumno> resultado = repositorio.findByCurso("5º Primaria");

    assertEquals(2, resultado.size());
}
```

```
    assertTrue(resultado.stream().anyMatch(a -> a.getDni().equals("12345678A")));
}
```

El test prepara tres alumnos: dos en curso y uno en otroCurso. Luego llama a findByCurso("5º Primaria") y verifica que devuelve dos. Si el método devolviera tres o uno, el test fallaría.

Un caso importante es probar el caso vacío: qué devuelve el método cuando no hay resultados. Normalmente debe devolver una lista vacía, no null.

```
@Test
void findByCurso_debeDevolverListaVacía_cuandoNoHayAlumnos() {
    List<Alumno> resultado = repositorio.findByCurso("Curso Inexistente");
    assertTrue(resultado.isEmpty());
}
```

T3.2 - Probar consultas con @Query

Las consultas con @Query se prueban igual, pero hay que prestar atención a dos cosas:

- Los parámetros. Hay que pasar valores concretos y verificar que el filtro se aplica correctamente.
- Los resultados. Pueden ser entidades, DTOs o Object[]. Cada uno se verifica de forma distinta.

```
@Test
void buscarPorCursoYNacidosDespues_debeFiltrarCorrectamente() {
    LocalDate fecha = LocalDate.of(2010, 1, 1);
    entityManager.persistAndFlush(
        new Alumno(
            "Ana", "García", "12345678A",
            LocalDate.of(2010, 5, 12), curso));
    entityManager.persistAndFlush(
        new Alumno(
            "Luis", "Pérez", "87654321B",
            LocalDate.of(2009, 9, 3), curso));
    entityManager.persistAndFlush(
        new Alumno(
            "María", "López", "11111111C",
            LocalDate.of(2011, 3, 20), curso));

    List<Alumno> resultado = repositorio.buscarPorCursoYNacidosDespues(
        "5º Primaria", fecha);

    assertEquals(2, resultado.size());
    assertTrue(
        resultado.stream()
            .allMatch(a -> a.getFechaNacimiento().isAfter(fecha)));
}
```

El test prepara tres alumnos: uno nacido en 2009, otro en 2010, otro en 2011. La consulta busca los nacidos después del 1 de enero de 2010. Debe devolver dos (2010 y 2011). Si devolviera tres, el filtro no se estaría aplicando.

Para consultas con proyección a DTO, se verifica el contenido del DTO:

```
@Test
void buscarResumenPorCurso_debeDevolverDTOs() {
    entityManager.persistAndFlush(
        new Alumno("Ana", "García", "12345678A", null, curso));

    List<AlumnoResumenDTO> resultado =
```

```

        repositorio.buscarResumenPorCurso("5º Primaria");

        assertEquals(1, resultado.size());
        assertEquals("Ana García", resultado.get(0).getNombreCompleto());
        assertEquals("5º Primaria", resultado.get(0).getCurso());
    }

```

T3.3 - Probar Specifications

Las Specifications se prueban combinando filtros y verificando que cada combinación devuelve lo esperado. Es más laborioso que un método derivado, pero importante porque las Specifications se construyen dinámicamente y son más propensas a errores.

```

@Test
void buscarConFiltros_debeFiltrarPorCursoYEstado() {
    entityManager.persistAndFlush(
        new Alumno("Ana", "García", "12345678A", null, curso));
    entityManager.persistAndFlush(
        new Alumno("Luis", "Pérez", "87654321B", null, otroCurso));
    Specification<Alumno> spec = Specification
        .where(AlumnoSpecifications.porCurso("5º Primaria"))
        .and(AlumnoSpecifications.porEstado(EstadoAlumno.ACTIVO));

    List<Alumno> resultado = repositorio.findAll(spec);

    assertEquals(1, resultado.size());
    assertEquals("Ana", resultado.get(0).getNombre());
}

```

Es importante probar varias combinaciones:

- Solo un filtro.
- Dos filtros combinados.
- Todos los filtros.
- Ningún filtro (devuelve todos).

Y probar el caso en el que un filtro es null: la Specification debe ignorarlo, no filtrar por null.

Pregunta de reflexión: ¿Por qué es importante probar el caso vacío en un método derivado? ¿Qué error detecta?

Bloque 4 - Probar relaciones y transacciones

T4.1 - Probar relaciones @ManyToOne y @OneToMany

Probar relaciones es importante porque el mapeo puede fallar de formas sutiles. Un mappedBy mal puesto, un FetchType incorrecto, un cascade mal configurado... todo eso se detecta con tests.

```

@Test
void guardarAlumnoConCurso_debePersistirLaRelacion() {
    Curso curso = entityManager.persistAndFlush(new Curso("5º Primaria"));
    Alumno alumno = new Alumno("Ana", "García", "12345678A", null);
    alumno.setCurso(curso);

    Alumno guardado = repositorio.save(alumno);
    entityManager.flush();
    entityManager.clear();

    Alumno recuperado = repositorio.findById(guardado.getId()).orElseThrow();
    assertNotNull(recuperado.getCurso());
}

```

```
    assertEquals("5º Primaria", recuperado.getCurso().getNombre());
}
```

El test hace varias cosas importantes:

- `entityManager.flush()` fuerza el INSERT en la base de datos.
- `entityManager.clear()` limpia el contexto de persistencia. Esto garantiza que la siguiente consulta va a la base de datos, no al caché de Hibernate.
- `repositorio.findById(...)` recupera la entidad desde la base de datos. Si la relación no se hubiera persistido, `getCurso()` devolvería null.

`clear()` es esencial cuando se prueba persistencia: sin él, Hibernate podría devolver la entidad del caché, que tiene la relación en memoria, sin comprobar si se guardó en la base de datos.

Para probar la relación `@OneToMany`:

```
@Test
void guardarCursoConAlumnos_debePersistirLaRelacion() {
    Curso curso = new Curso("5º Primaria");
    curso.addAlumno(new Alumno("Ana", "García", "12345678A", null));
    curso.addAlumno(new Alumno("Luis", "Pérez", "87654321B", null));
    entityManager.persistAndFlush(curso);
    entityManager.clear();

    Curso recuperado = entityManager.find(Curso.class, curso.getId());
    assertEquals(2, recuperado.getAlumnos().size());
}
```

T4.2 - Probar FetchType.LAZY

Verificar que una relación LAZY no se carga hasta que se accede es complicado en un test, porque `@DataJpaTest` envuelve el test en una transacción, y dentro de la transacción el `LazyInitializationException` no se produce.

Para probar el comportamiento de LAZY, hay que salir de la transacción. Se hace con `@Transactional(propagation = Propagation.NOT_SUPPORTED)` en el método de test:

```
@Test
@Transactional(propagation = Propagation.NOT_SUPPORTED)
void lazyLoading_debeFallarFueraDeTransaccion() {
    // Preparar datos fuera de la transacción
    Curso curso = cursoRepository.save(new Curso("5º Primaria"));
    Alumno alumno = new Alumno("Ana", "García", "12345678A", null);
    alumno.setCurso(curso);
    alumnoRepository.save(alumno);

    // Cargar el alumno (transacción cerrada después del save)
    Alumno recuperado = alumnoRepository.findById(alumno.getId()).orElseThrow();

    // Acceder a la relación LAZY debe lanzar LazyInitializationException
    assertThrows(
        LazyInitializationException.class,
        () -> recuperado.getCurso().getNombre());

    // Limpiar
    alumnoRepository.deleteAll();
    cursoRepository.deleteAll();
}
```

Nota: como el test no está envuelto en transacción, hay que limpiar los datos manualmente al final. Es un caso particular.

En la práctica, este test es útil cuando quieres verificar que el FetchType es LAZY. Si alguien cambia el FetchType a EAGER, el test falla. Es una forma de proteger la configuración.

T4.3 - Probar transacciones y rollback

Probar el rollback de una transacción es importante para garantizar que las operaciones son atómicas. Se hace verificando que, tras un fallo, los datos no se han persistido.

```
@Test
void crearTransaccional_debeHacerRollback_cuandoFalla() {
    // Este test verifica que si el servicio falla después de guardar,
    // el guardado se revierte.
    assertThrows(RuntimeException.class, () -> {
        // Simular la llamada a un método transaccional que falla
        Curso curso = cursoRepository.save(new Curso("5º Primaria"));
        Alumno alumno = new Alumno("Ana", "García", "12345678A", null);
        alumno.setCurso(curso);
        alumnoRepository.save(alumno);
        throw new RuntimeException("Error simulado");
    });
    // Verificar que no hay datos persistidos
    assertEquals(0, alumnoRepository.count());
}
```

Ojo: este test funciona dentro de la transacción de @DataJpaTest, que se revierte al final. Para probar el rollback real de un método @Transactional, hay que usar @Transactional(propagation = Propagation.NOT_SUPPORTED) en el test y llamar al servicio, no al repositorio.

En un test de JPA puro, el caso más típico es verificar que las restricciones de la base de datos se aplican:

```
@Test
void guardarAlumnoConDniDuplicado_debeLanzarExcepcion() {
    entityManager.persistAndFlush(
        new Alumno("Ana", "García", "12345678A", null, curso));
    Alumno duplicado = new Alumno("Otro", "Alumno", "12345678A", null, curso);

    assertThrows(DataIntegrityViolationException.class, () -> {
        repositorio.saveAndFlush(duplicado);
    });
}
```

DataIntegrityViolationException es la excepción que Spring lanza cuando la base de datos detecta una violación de constraint (DNI duplicado, nullable, unique...). El test verifica que la restricción unique = true del campo dni se aplica.

Pregunta de reflexión: ¿Por qué se usa entityManager.clear() antes de verificar una persistencia? ¿Qué pasaría sin él?

Bloque 5 - Buenas prácticas y errores

T5.1 - Buenas prácticas en tests de JPA

Un test por comportamiento. Igual que en los tests unitarios. Cada test verifica una cosa.

Nombres descriptivos. findByCurso_debeDevolverAlumnosDelCurso es mejor que test1. El formato metodo_condicion_resultado es el estándar.

Usar `TestEntityManager` para preparar datos. Es más limpio que usar el repositorio.

Usar `entityManager.clear()` cuando se prueba persistencia. Garantiza que se lee de la base de datos, no del caché.

Usar `persistAndFlush` en lugar de `persist`. El flush garantiza que la entidad tiene ID y que la base de datos tiene el estado consistente.

Probar el caso vacío. Un método que devuelve una lista debe devolver [], no null.

Probar los constraints. Las restricciones de la base de datos son parte del contrato. Hay que verificar que se aplican.

No probar la lógica de negocio en `@DataJpaTest`. Solo el acceso a datos.

No usar `@SpringBootTest` para probar repositorios. Es más lento y carga todo sin necesidad.

Limpiar los datos manualmente cuando el test no esté envuelto en transacción. Si has usado `NOT_SUPPORTED`, hay que limpiar.

T5.2 - Errores comunes

Error 1: usar `@SpringBootTest` para probar el repositorio.

`@SpringBootTest` arranca toda la aplicación. Es más lento y carga beans innecesarios. Para repositorios, usar `@DataJpaTest`.

Error 2: no usar `entityManager.clear()` al probar persistencia.

Sin `clear()`, Hibernate devuelve la entidad del caché, que tiene los datos en memoria. El test puede pasar aunque la persistencia esté mal.

Error 3: olvidar `persistAndFlush` en lugar de `persist`.

Con `persist`, la entidad no se ha escrito en la base de datos todavía. Si el test hace una consulta, Hibernate puede que no la encuentre (depende del flush automático).

Error 4: usar `@Transactional` en el test sin necesidad.

`@DataJpaTest` ya incluye transacción. Añadir `@Transactional` explícitamente es redundante. Solo se usa `NOT_SUPPORTED` para desactivarla.

Error 5: no probar los constraints.

Si no pruebas que un DNI duplicado falla, un cambio en la entidad podría eliminar la restricción sin que te enteres.

Error 6: compartir estado entre tests. Si los tests no están aislados (por ejemplo, con `NOT_SUPPORTED`), el estado de un test afecta al siguiente. Hay que limpiar entre tests.

Error 7: no usar `TestEntityManager`.

Usar el repositorio directamente en el `@BeforeEach` funciona, pero es menos flexible. `TestEntityManager` está diseñado para eso.

Error 8: olvidar que `@DataJpaTest` no carga servicios ni controladores.

Si el test intenta inyectar un servicio, falla. Para eso, usar `@SpringBootTest` o `@Import` para importar el bean específico.

Error 9: asumir que el orden de los resultados es estable.

Si no hay `ORDER BY`, la base de datos puede devolver los resultados en cualquier orden. Los tests deben usar `anyMatch` o verificar el conjunto, no el orden.

Error 10: usar datos de producción en tests.

Los tests siempre usan datos sintéticos y una base de datos en memoria. Nunca la de producción.

T5.3 - Cuándo usar @DataJpaTest y cuándo @SpringBootTest

La elección entre @DataJpaTest y @SpringBootTest depende de qué se quiere probar:

@DataJpaTest:

- Probar repositorios.
- Probar entidades y su mapeo.
- Probar consultas JPQL.
- Probar Specifications.
- Probar relaciones y cascade.
- Probar transacciones a nivel de repositorio.
- Ventaja: rápido, aislado, con H2 en memoria.
- Inconveniente: no carga servicios ni controladores.

@SpringBootTest:

- Probar el flujo completo: controlador → servicio → repositorio.
- Probar la integración entre capas.
- Probar la configuración de la aplicación.
- Probar la seguridad.
- Ventaja: prueba todo el contexto.
- Inconveniente: más lento, arranca todo.

Regla: usar @DataJpaTest para tests de repositorios y @SpringBootTest para tests de integración completa. La mayoría de los tests de JPA son del primer tipo.

Pregunta de reflexión: ¿Por qué @DataJpaTest es más rápido que @SpringBootTest?
¿Qué deja de cargar?

Resumen de la teoría

- Testing de JPA: usa una base de datos real (H2 en memoria), no mocks.
- @DataJpaTest: arranca solo la infraestructura JPA. Rápido y aislado.
- Transacción por test: cada test se ejecuta en una transacción que se revierte al final.
- TestEntityManager: clase de ayuda para preparar datos.
- persistAndFlush: persistir y forzar el flush.
- clear(): limpiar el contexto para forzar una lectura de la base de datos.
- Probar consultas derivadas, @Query, Specifications.
- Probar relaciones: @ManyToOne, @OneToMany, FetchType.LAZY.
- Probar constraints: DataIntegrityViolationException para DNI duplicado.
- @DataJpaTest vs @SpringBootTest: repositorios vs integración completa.
- Buenas prácticas: nombres descriptivos, caso vacío, constraints, clear().

Criterio profesional añadido

En Spring Boot 3.x la persistencia usa el espacio de nombres jakarta.persistence. La transacción pertenece a la capa de servicio, las entidades no deben ser el contrato HTTP y cualquier consulta potencialmente costosa debe comprobarse con SQL/logs y pruebas de repositorio.